



Universidade Estadual de Santa Cruz – UESC

Relatório de Implementações de Métodos da Disciplina Análise Numérica

**Relatório de implementações
realizadas por Davi Roriz Oliveira**

Disciplina: Análise Numérica

Curso: Ciência da Computação

Semestre 2024.1

Professor: Gesil Sampaio Amarante II

**Ilhéus – BA
2024**

Índice

1. Linguagem escolhida e justificativas.....	3
2. Método de Euler.....	4
3. Método de Heun.....	12
4. Método de Euler Modificado.....	20
5. Método de Ralston.....	28
6. Método de Hunge-Kutta de 3 ordem.....	36
7. Método de Hunge-Kutta de 4 ordem.....	44
8. Método dos Sistemas de Equações diferenciais.....	42
9. Método de Shooting.....	55
10. Método das Diferenças Finitas.....	58
11. Comparação de resultados.....	61
12. Considerações finais.....	64

1. Linguagem escolhida e justificativas

Para a realização do projeto, foi utilizado a linguagem de programação Python na sua versão 3.11.2, sua escolha deve-se ao fato da facilidade na implementação de códigos para cálculos matemáticos. Pois, a linguagem possui uma ampla variedade de bibliotecas que auxiliam a codificação de algoritmos com estes.

As bibliotecas Sympy, Math e Numpy (as duas últimas não foram utilizadas neste relatório), tem como objetivo justamente fornecer ferramentas de álgebra computacional para os programas. Também temos a biblioteca Os, que foi utilizada para acessar o diretório em que os arquivos se encontram e poder abrir os arquivos de entrada e saída. Por fim, Matplotlib que foi necessária para plotar os gráficos fornecidos no projeto.

Além de serem de fácil instalação, a linguagem também é multiplataforma, podendo operar em diferentes sistemas operacionais, bem como Windows, Linux e MacOS. Sua sintaxe é simples comparada a outras linguagens, possibilitando que programadores de diferentes níveis de conhecimento possam aprender e manipular códigos em Python, bem como compreender as implementações dos códigos deste projeto.

2. Método de Euler

Estratégia de Implementação:

O arquivo contém 4 funções principais: `main()`, `grafico()`, `solucao()` e `euler()`.

A função `main()` tem como objetivo encontrar o diretório em que a pasta do arquivo se encontra, e definir o arquivo de entrada “in.txt” e o arquivo “out.txt”. Após isso, ela faz a leitura dos dados do arquivo de entrada e faz a chamada das demais funções do algoritmo para a realização dos cálculos necessários para o método. Por fim, o resultado gerado é posto no arquivo de saída.

A função `euler()` tem como objetivo fazer o cálculo do método e gerar a sua aproximação. Ela tem como parâmetros a função “f”, e os valores de “x0”, y0”, “h” e “n”.

Por último, a função `grafico()` recebe a função, e os valores de “x0”, y0”, “h” e “n”, e plota o gráfico que contém o conjunto de dados e a solução correta para o problema. Para a realização do gráfico, foi necessário utilizar a biblioteca Matplotlib.

Estrutura dos arquivos de entrada / saída:

A estrutura do arquivo de entrada “in.txt” consiste em 5 linhas, a primeira contendo a função, seguido do x0, y0, h e n, respectivamente, cada um em uma linha

O arquivo de saída “out.txt” consiste em $3 * (n + 1)$ linhas, para cada divisão n (pontos internos), ele irá printar os valores de x e y em uma linha, o y exato na outra, e a taxa de erro na terceira linha, repetindo-se para todos os pontos internos.

Problema teste:

Para a realização do método, foram testados dois problemas apresentados nos slides do professor (Problema 1 e Problema 2). Além destes 2 exercícios da questão 12.2 da página 389 do livro Neide Franco (Problema 3 e Problema 4:

12.2 - Considere os seguintes problemas de valor inicial:

$$a) \begin{cases} y' = y^2 + 1 \\ y(0) = 0 \end{cases} ; 0 \leq x \leq 1 ; h = 0.2$$

$$b) \begin{cases} y' = -2xy \\ y(0) = 1 \end{cases} ; 0 \leq x \leq 0.6 ; h = 0.3$$

Obs: A quantidade de pontos internos foram aumentados para esses 2 problemas.

1. Problema 1:

Entrada:

$$f(x) = 4*(E)**(0.8*x) - 0.5*y$$

$$x0 = 0$$

$$y0 = 2$$

$$h = 1$$

$$n = 4$$

Resultado Obtido:

$$x: 0.0 // y: 2.0$$

$$y_exato: 2.0000000000000000$$

$$\text{Erro: } 0\%$$

$$x: 1.0 // y: 5.0$$

$$y_exato: 6.19463137720937$$

$$\text{Erro: } -19.28\%$$

$$x: 2.0 // y: 11.402163713969871$$

$$y_exato: 14.8439219076465$$

$$\text{Erro: } -23.19\%$$

$$x: 3.0 // y: 25.513211554565395$$

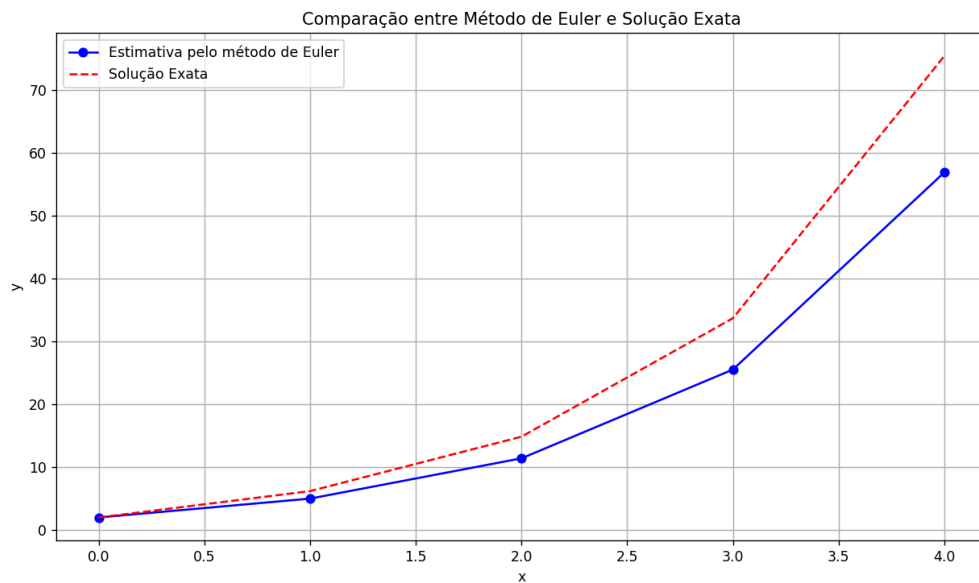
y_exato: 33.6771717679682

Erro: -24.24%

x: 4.0 // y: 56.84931129984912

y_exato: 75.3389626091586

Erro: -24.54%



2. Problema 2:

Entrada:

$$f(x) = -2x^3 + 12x^2 - 20x + 8.5$$

$$x_0 = 0$$

$$y_0 = 1$$

$$h = 0.5$$

$$n = 8$$

Resultado Obtido:

x: 0.0 // y: 1.0

y_exato: 1.0000000000000000

Erro: 0%

x: 0.5 // y: 5.25

y_exato: 3.218750000000000

Erro: 63.11%

x: 1.0 // y: 5.875

y_exato: 3.000000000000000

Erro: 95.83%

x: 1.5 // y: 5.125

y_exato: 2.218750000000000

Erro: 130.99%

x: 2.0 // y: 4.5

y_exato: 2.000000000000000

Erro: 125.0000000000000%

x: 2.5 // y: 4.75

y_exato: 2.718750000000000

Erro: 74.71%

x: 3.0 // y: 5.875

y_exato: 4.000000000000000

Erro: 46.88%

x: 3.5 // y: 7.125

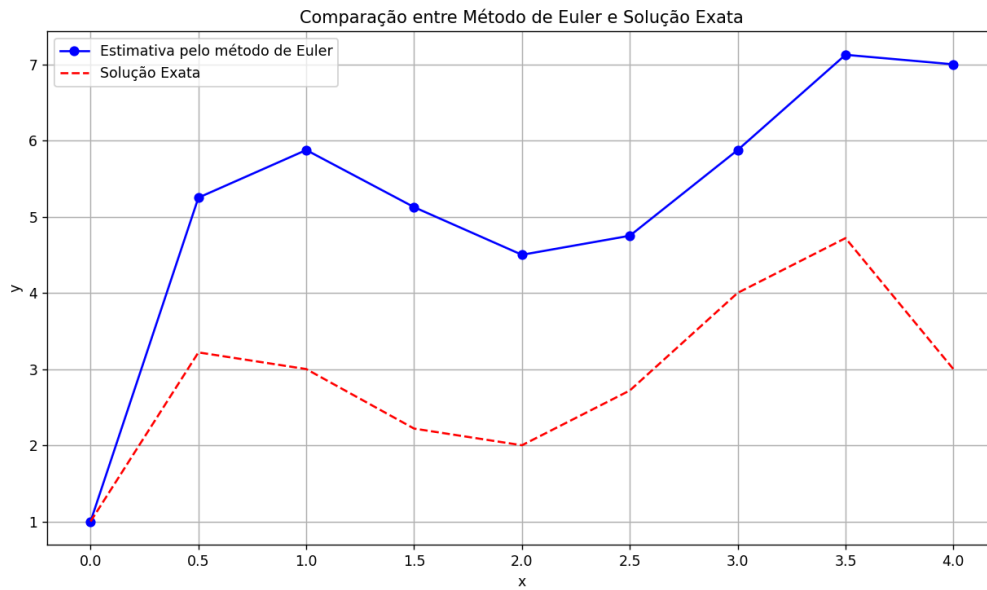
y_exato: 4.718750000000000

Erro: 50.99%

x: 4.0 // y: 7.0

y_exato: 3.000000000000000

Erro: 133.33%



3. Problema 3:

Entrada:

$$f(x) = y^2 + 1$$

$$x_0 = 0$$

$$y_0 = 0$$

$$h = 0.2$$

$$n = 5$$

Resultado Obtido:

x: 0.0 // y: 0.0

y_exato: 0

Erro: nan%

x: 0.2 // y: 0.2

y_exato: 0.202710035508673

Erro: -1.34%

x: 0.4 // y: 0.40800000000000003

y_exato: 0.422793218738162

Erro: -3.50%

x: 0.6000000000000001 // y: 0.6412928

y_exato: 0.684136808341692

Erro: -6.26%

x: 0.8 // y: 0.923544091066368

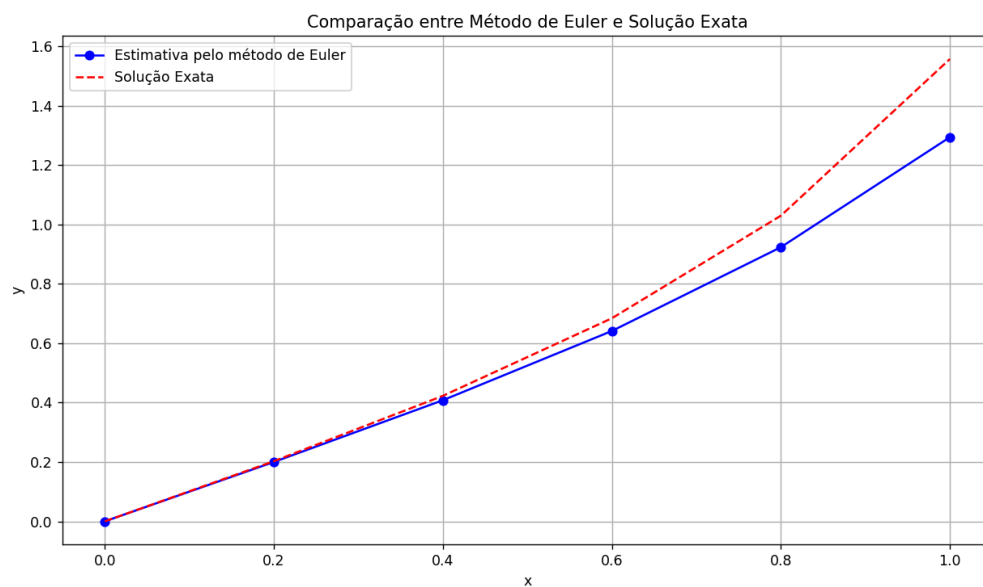
y_exato: 1.02963855705036

Erro: -10.30%

x: 1.0 // y: 1.294130828695089

y_exato: 1.55740772465490

Erro: -16.90%



4. Problema 4:

Entrada:

$$f(x) = -2 \cdot x \cdot y$$

$$x_0 = 0$$

$$y_0 = 1$$

$$h = 0.3$$

$$n = 5$$

Resultado Obtido:

x: 0.0 // y: 1.0

y_exato: 1.000000000000000

Erro: 0%

x: 0.3 // y: 1.0

y_exato: 0.913931185271228

Erro: 9.42%

x: 0.6 // y: 0.8200000000000001

y_exato: 0.697676326071031

Erro: 17.53%

x: 0.8999999999999999 // y: 0.5248000000000002

y_exato: 0.444858066222941

Erro: 17.97%

x: 1.2 // y: 0.24140800000000012

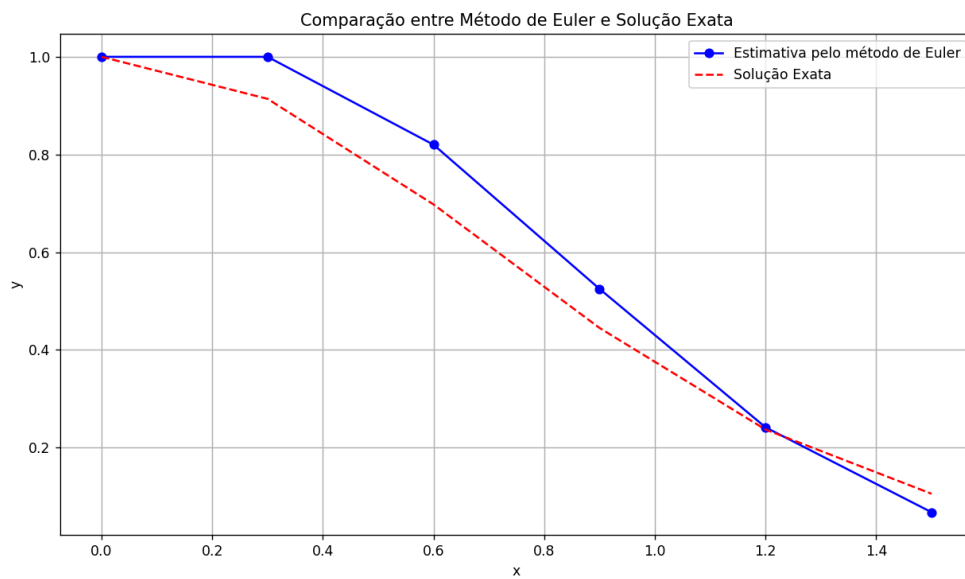
y_exato: 0.236927758682122

Erro: 1.89%

x: 1.5 // y: 0.06759424000000003

y_exato: 0.105399224561864

Erro: -35.87%



Dificuldades enfrentadas:

Não houve dificuldades enfrentadas.

3. Método de Heun

Estratégia de Implementação:

O arquivo contém 4 funções principais: `main()`, `grafico()`, `solucao()` e `heun()`.

A função `main()` tem como objetivo encontrar o diretório em que a pasta do arquivo se encontra, e definir o arquivo de entrada “in.txt” e o arquivo “out.txt”. Após isso, ela faz a leitura dos dados do arquivo de entrada e faz a chamada das demais funções do algoritmo para a realização dos cálculos necessários para o método. Por fim, o resultado gerado é posto no arquivo de saída.

A função `heun()` tem como objetivo fazer o cálculo do método e gerar a sua aproximação. Ela tem como parâmetros a função “f”, e os valores de “x0”, y0”, “h” e “n”.

Por último, a função `grafico()` recebe a função, e os valores de “x0”, y0”, “h” e “n”, e plota o gráfico que contém o conjunto de dados e a solução correta para o problema. Para a realização do gráfico, foi necessário utilizar a biblioteca Matplotlib.

Estrutura dos arquivos de entrada / saída:

A estrutura do arquivo de entrada “in.txt” consiste em 5 linhas, a primeira contendo a função, seguido do x0, y0, h e n, respectivamente, cada um em uma linha

O arquivo de saída “out.txt” consiste em $3 * (n + 1)$ linhas, para cada divisão n (pontos internos), ele irá printar os valores de x e y em uma linha, o y exato na outra, e a taxa de erro na terceira linha, repetindo-se para todos os pontos internos.

Problema teste:

Para a realização do método, foram testados dois problemas apresentados nos slides do professor (Problema 1 e Problema 2). Além destes 2 exercícios da questão 12.2 da página 389 do livro Neide Franco (Problema 3 e Problema 4:

12.2 - Considere os seguintes problemas de valor inicial:

$$a) \begin{cases} y' = y^2 + 1 \\ y(0) = 0 \end{cases} ; \quad 0 \leq x \leq 1 ; \quad h = 0.2$$

$$b) \begin{cases} y' = -2xy \\ y(0) = 1 \end{cases} ; \quad 0 \leq x \leq 0.6 ; \quad h = 0.3$$

Obs: A quantidade de pontos internos foram aumentados para esses 2 problemas.

5. Problema 1:

Entrada:

$$f(x) = 4*(E)**(0.8*x) - 0.5*y$$

$$x_0 = 0$$

$$y_0 = 2$$

$$h = 1$$

$$n = 4$$

Resultado Obtido:

$$x: 0.0, y: 2.0$$

$$y_{\text{exato}}: 2.0000000000000000$$

$$\text{Erro: } 0\%$$

$$x: 1.0, y: 6.701081856984936$$

$$y_{\text{exato}}: 6.19463137720937$$

$$\text{Erro: } 8.18\%$$

$$x: 2.0, y: 16.31978193789828$$

$$y_{\text{exato}}: 14.8439219076465$$

$$\text{Erro: } 9.94\%$$

$$x: 3.0, y: 37.199248896864745$$

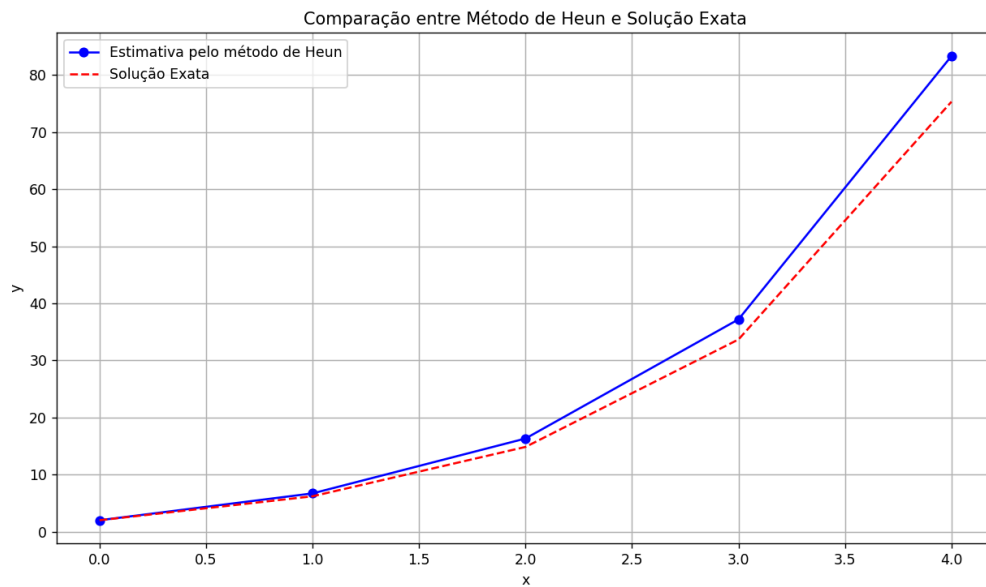
y_exato: 33.6771717679682

Erro: 10.46%

x: 4.0, y: 83.33776733540077

y_exato: 75.3389626091586

Erro: 10.62%



6. Problema 2:

Entrada:

$$f(x) = -2x^3 + 12x^2 - 20x + 8.5$$

$$x_0 = 0$$

$$y_0 = 1$$

$$h = 0.5$$

$$n = 8$$

Resultado Obtido:

x: 0.0, y: 1.0

y_exato: 1.0000000000000000

Erro: 0%

x: 0.5, y: 3.4375

y_exato: 3.218750000000000

Erro: 6.80%

x: 1.0, y: 3.375

y_exato: 3.000000000000000

Erro: 12.50%

x: 1.5, y: 2.6875

y_exato: 2.218750000000000

Erro: 21.13%

x: 2.0, y: 2.5

y_exato: 2.000000000000000

Erro: 25.000000000000000%

x: 2.5, y: 3.1875

y_exato: 2.718750000000000

Erro: 17.24%

x: 3.0, y: 4.375

y_exato: 4.000000000000000

Erro: 9.38%

x: 3.5, y: 4.9375

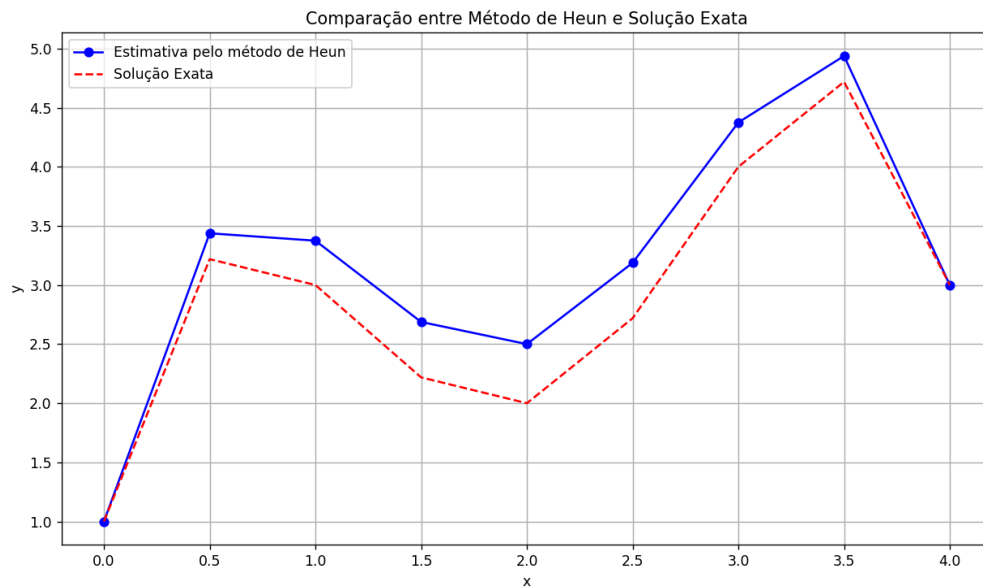
y_exato: 4.718750000000000

Erro: 4.64%

x: 4.0, y: 3.0

y_exato: 3.000000000000000

Erro: 0%



7. Problema 3:

Entrada:

$$f(x) = y^2 + 1$$

$$x_0 = 0$$

$$y_0 = 0$$

$$h = 0.2$$

$$n = 5$$

Resultado Obtido:

x: 0.0, y: 0.0

y_exato: 0

Erro: nan%

x: 0.2, y: 0.20400000000000001

y_exato: 0.202710035508673

Erro: 0.64%

x: 0.4, y: 0.42516264212582405

y_exato: 0.422793218738162

Erro: 0.56%

x: 0.6000000000000001, y: 0.6869727615005312

y_exato: 0.684136808341692

Erro: 0.41%

x: 0.8, y: 1.0304724827096094

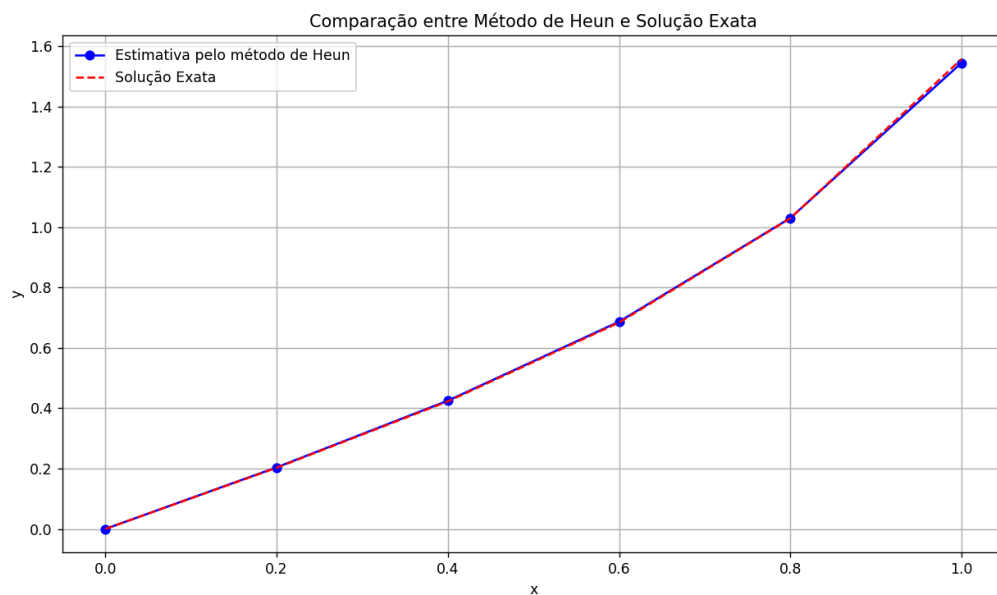
y_exato: 1.02963855705036

Erro: 0.08%

x: 1.0, y: 1.5448406379083808

y_exato: 1.55740772465490

Erro: -0.81%



8. Problema 4:

Entrada:

$$f(x) = -2 \cdot x \cdot y$$

$$x_0 = 0$$

$$y_0 = 1$$

$$h = 0.3$$

$$n = 5$$

Resultado Obtido:

x: 0.0, y: 1.0

y_exato: 1.000000000000000

Erro: 0%

x: 0.3, y: 0.91

y_exato: 0.913931185271228

Erro: -0.43%

x: 0.6, y: 0.6937840000000001

y_exato: 0.697676326071031

Erro: -0.56%

x: 0.8999999999999999, y: 0.44901700480000006

y_exato: 0.444858066222941

Erro: 0.93%

x: 1.2, y: 0.25342519750912007

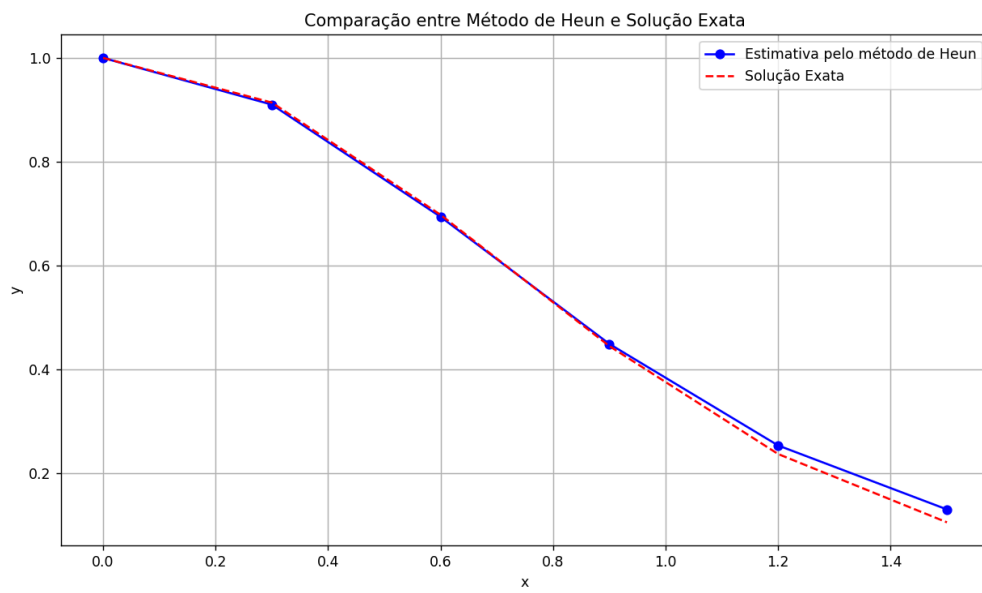
y_exato: 0.236927758682122

Erro: 6.96%

x: 1.5, y: 0.1302605515196877

y_exato: 0.105399224561864

Erro: 23.59%



Dificuldades enfrentadas:

Não houve dificuldades enfrentadas.

4. Método de Euler Modificado

Estratégia de Implementação:

O arquivo contém 4 funções principais: `main()`, `grafico()`, `solucao()` e `euler_modificado()`.

A função `main()` tem como objetivo encontrar o diretório em que a pasta do arquivo se encontra, e definir o arquivo de entrada “in.txt” e o arquivo “out.txt”. Após isso, ela faz a leitura dos dados do arquivo de entrada e faz a chamada das demais funções do algoritmo para a realização dos cálculos necessários para o método. Por fim, o resultado gerado é posto no arquivo de saída.

A função `euler_modificado()` tem como objetivo fazer o cálculo do método e gerar a sua aproximação. Ela tem como parâmetros a função “f”, e os valores de “x0”, y0”, “h” e “n”.

Por último, a função `grafico()` recebe a função, e os valores de “x0”, y0”, “h” e “n”, e plota o gráfico que contém o conjunto de dados e a solução correta para o problema. Para a realização do gráfico, foi necessário utilizar a biblioteca Matplotlib.

Estrutura dos arquivos de entrada / saída:

A estrutura do arquivo de entrada “in.txt” consiste em 5 linhas, a primeira contendo a função, seguido do x0, y0, h e n, respectivamente, cada um em uma linha

O arquivo de saída “out.txt” consiste em $3 * (n + 1)$ linhas, para cada divisão n (pontos internos), ele irá printar os valores de x e y em uma linha, o y exato na outra, e a taxa de erro na terceira linha, repetindo-se para todos os pontos internos.

Problema teste:

Para a realização do método, foram testados dois problemas apresentados nos slides do professor (Problema 1 e Problema 2). Além destes 2 exercícios da questão 12.2 da página 389 do livro Neide Franco (Problema 3 e Problema 4:

12.2 - Considere os seguintes problemas de valor inicial:

$$a) \begin{cases} y' = y^2 + 1 \\ y(0) = 0 \end{cases} ; \quad 0 \leq x \leq 1 \quad ; \quad h = 0.2$$

$$b) \begin{cases} y' = -2xy \\ y(0) = 1 \end{cases} ; \quad 0 \leq x \leq 0.6 \quad ; \quad h = 0.3$$

Obs: A quantidade de pontos internos foram aumentados para esses 2 problemas.

9. Problema 1:

Entrada:

$$f(x) = 4*(E)**(0.8*x) - 0.5*y$$

$$x_0 = 0$$

$$y_0 = 2$$

$$h = 1$$

$$n = 4$$

Resultado Obtido:

$$x: 0.0, y: 2.0$$

$$y_{\text{exato}}: 2.0000000000000000$$

$$\text{Erro: } 0\%$$

$$x: 1.0, y: 6.217298790565081$$

$$y_{\text{exato}}: 6.19463137720937$$

$$\text{Erro: } 0.37\%$$

$$x: 2.0, y: 14.940738506556901$$

$$y_{\text{exato}}: 14.8439219076465$$

$$\text{Erro: } 0.65\%$$

$$x: 3.0, y: 33.941153537925544$$

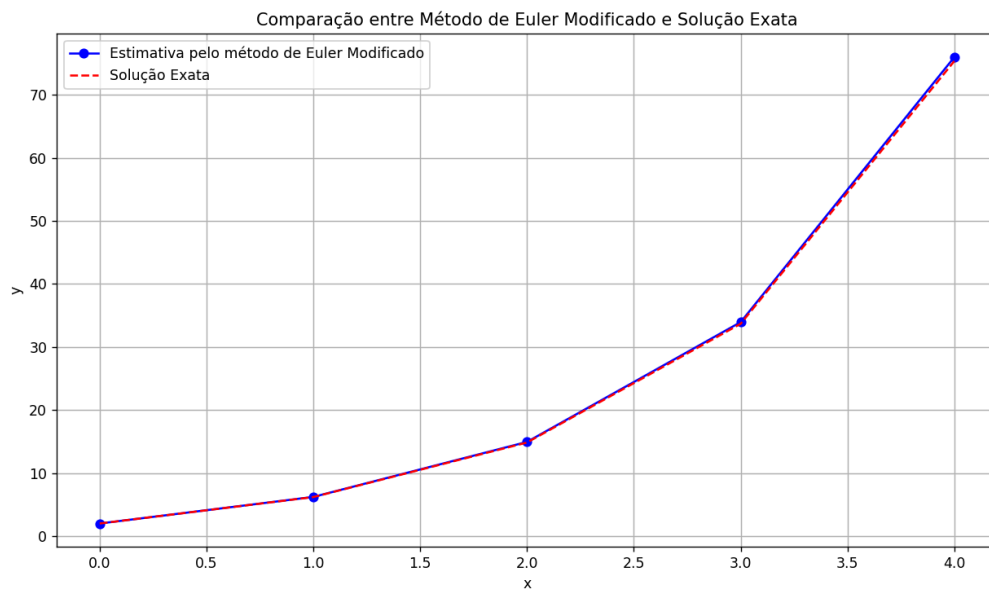
y_exato: 33.6771717679682

Erro: 0.78%

x: 4.0, y: 75.96863166495008

y_exato: 75.3389626091586

Erro: 0.84%



10.Problema 2:

Entrada:

$$f(x) = -2*x**3 + 12*x**2 - 20*x + 8.5$$

$$x0 = 0$$

$$y0 = 1$$

$$h = 0.5$$

$$n = 8$$

Resultado Obtido:

$$x: 0.0, y: 1.0$$

$$y_exato: 1.0000000000000000$$

$$\text{Erro: } 0\%$$

$$x: 0.5, y: 3.109375$$

y_exato: 3.218750000000000

Erro: -3.40%

x: 1.0, y: 2.8125

y_exato: 3.000000000000000

Erro: -6.25%

x: 1.5, y: 1.984375

y_exato: 2.218750000000000

Erro: -10.56%

x: 2.0, y: 1.75

y_exato: 2.000000000000000

Erro: -12.50%

x: 2.5, y: 2.484375

y_exato: 2.718750000000000

Erro: -8.62%

x: 3.0, y: 3.8125

y_exato: 4.000000000000000

Erro: -4.69%

x: 3.5, y: 4.609375

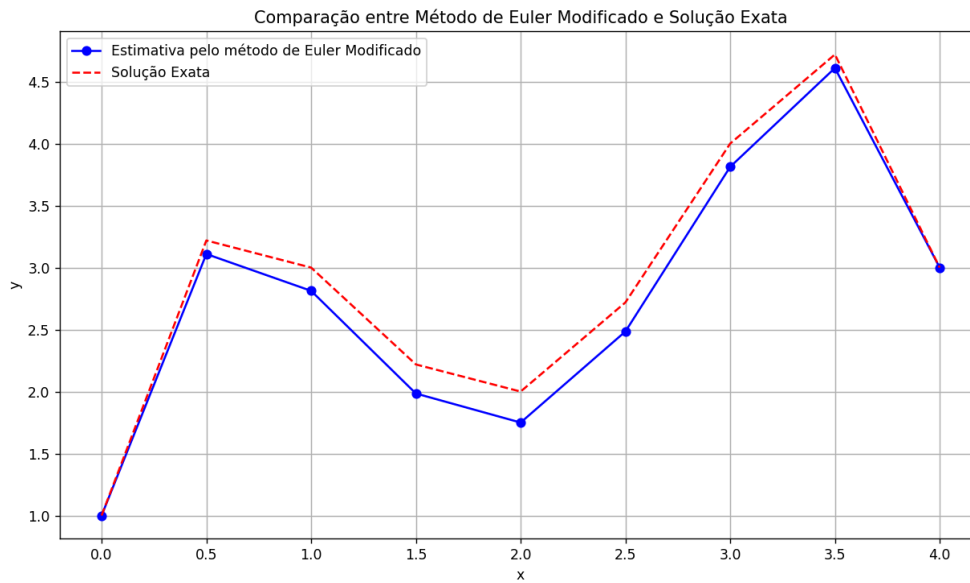
y_exato: 4.718750000000000

Erro: -2.32%

x: 4.0, y: 3.0

y_exato: 3.000000000000000

Erro: 0%



11.Problema 3:

Entrada:

$$f(x) = y^2 + 1$$

$$x_0 = 0$$

$$y_0 = 0$$

$$h = 0.2$$

$$n = 5$$

Resultado Obtido:

x: 0.0, y: 0.0

y_exato: 0

Erro: nan%

x: 0.2, y: 0.202

y_exato: 0.202710035508673

Erro: -0.35%

x: 0.4, y: 0.42073704225283204

y_exato: 0.422793218738162

Erro: -0.49%

x: 0.6000000000000001, y: 0.6787203553470016

y_exato: 0.684136808341692

Erro: -0.79%

x: 0.8, y: 1.0147749053141941

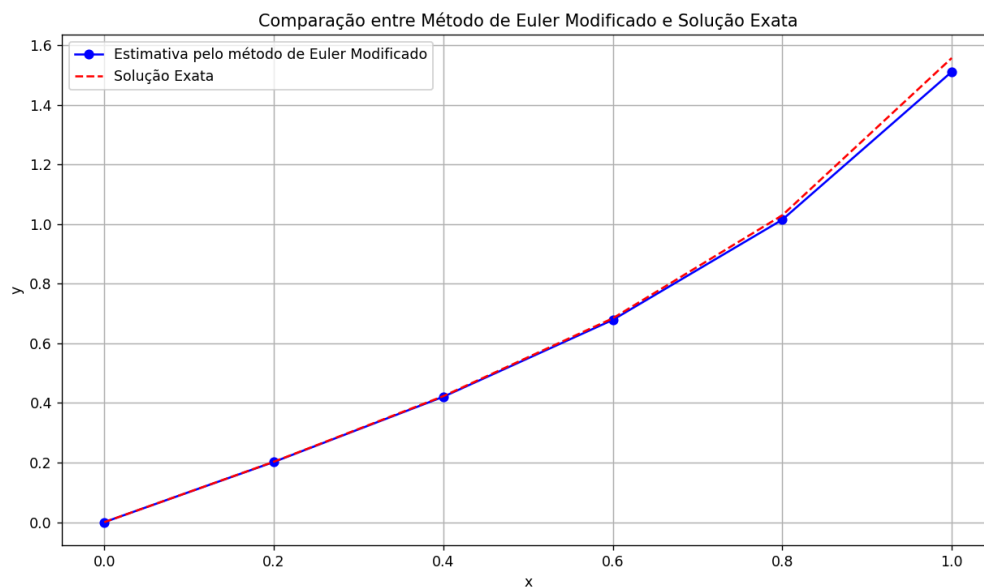
y_exato: 1.02963855705036

Erro: -1.44%

x: 1.0, y: 1.5113587537561912

y_exato: 1.55740772465490

Erro: -2.96%



12.Problema 4:

Entrada:

$$f(x) = -2*x*y$$

$$x_0 = 0$$

$$y_0 = 1$$

$$h = 0.3$$

$$n = 5$$

Resultado Obtido:

x: 0.0, y: 1.0

y_exato: 1.000000000000000

Erro: 0%

x: 0.3, y: 0.91

y_exato: 0.913931185271228

Erro: -0.43%

x: 0.6, y: 0.686413

y_exato: 0.697676326071031

Erro: -1.61%

x: 0.8999999999999999, y: 0.43312660300000005

y_exato: 0.444858066222941

Erro: -2.64%

x: 1.2, y: 0.23393167828030006

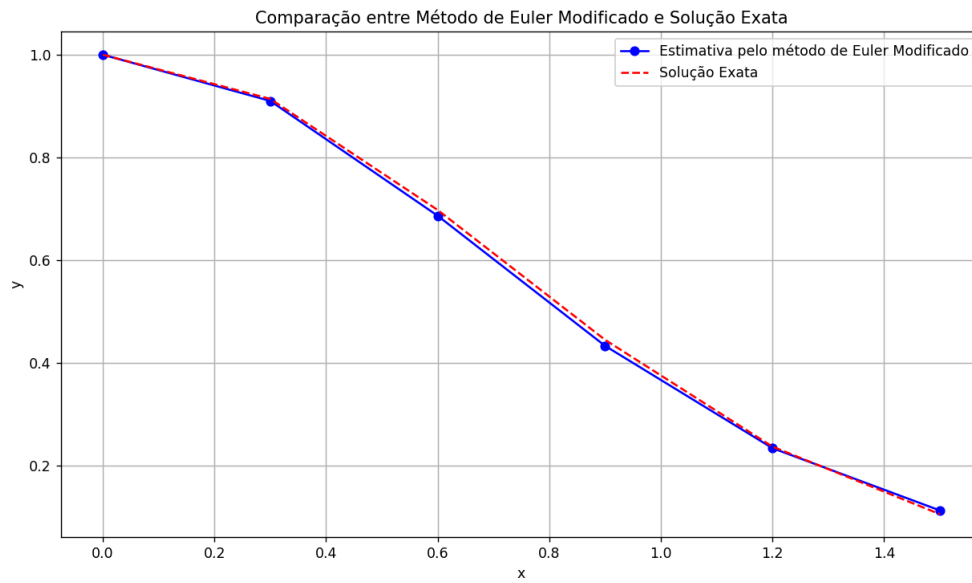
y_exato: 0.236927758682122

Erro: -1.26%

x: 1.5, y: 0.11266149625979253

y_exato: 0.105399224561864

Erro: 6.89%



Dificuldades enfrentadas:

Não houve dificuldades enfrentadas.

5. Método de Ralston

Estratégia de Implementação:

O arquivo contém 4 funções principais: `main()`, `grafico()`, `solucao()` e `ralston()`.

A função `main()` tem como objetivo encontrar o diretório em que a pasta do arquivo se encontra, e definir o arquivo de entrada “in.txt” e o arquivo “out.txt”. Após isso, ela faz a leitura dos dados do arquivo de entrada e faz a chamada das demais funções do algoritmo para a realização dos cálculos necessários para o método. Por fim, o resultado gerado é posto no arquivo de saída.

A função `ralston()` tem como objetivo fazer o cálculo do método e gerar a sua aproximação. Ela tem como parâmetros a função “f”, e os valores de “x0”, y0”, “h” e “n”.

Por último, a função `grafico()` recebe a função, e os valores de “x0”, y0”, “h” e “n”, e plota o gráfico que contém o conjunto de dados e a solução correta para o problema. Para a realização do gráfico, foi necessário utilizar a biblioteca Matplotlib.

Estrutura dos arquivos de entrada / saída:

A estrutura do arquivo de entrada “in.txt” consiste em 5 linhas, a primeira contendo a função, seguido do x0, y0, h e n, respectivamente, cada um em uma linha

O arquivo de saída “out.txt” consiste em $3 * (n + 1)$ linhas, para cada divisão n (pontos internos), ele irá printar os valores de x e y em uma linha, o y exato na outra, e a taxa de erro na terceira linha, repetindo-se para todos os pontos internos.

Problema teste:

Para a realização do método, foram testados dois problemas apresentados nos slides do professor (Problema 1 e Problema 2). Além destes 2 exercícios da questão 12.2 da página 389 do livro Neide Franco (Problema 3 e Problema 4:

12.2 - Considere os seguintes problemas de valor inicial:

$$a) \begin{cases} y' = y^2 + 1 \\ y(0) = 0 \end{cases} ; \quad 0 \leq x \leq 1 ; \quad h = 0.2$$

$$b) \begin{cases} y' = -2xy \\ y(0) = 1 \end{cases} ; \quad 0 \leq x \leq 0.6 ; \quad h = 0.3$$

Obs: A quantidade de pontos internos foram aumentados para esses 2 problemas.

13.Problema 1:

Entrada:

$$f(x) = 4*(E)**(0.8*x) - 0.5*y$$

$$x_0 = 0$$

$$y_0 = 2$$

$$h = 1$$

$$n = 4$$

Resultado Obtido:

$$x: 0.0, y: 2.0$$

$$y_{\text{exato}}: 2.0000000000000000$$

$$\text{Erro: } 0\%$$

$$x: 1.0, y: 6.442316801041358$$

$$y_{\text{exato}}: 6.19463137720937$$

$$\text{Erro: } 4.000000000000000\%$$

$$x: 2.0, y: 15.582161555067472$$

$$y_{\text{exato}}: 14.8439219076465$$

$$\text{Erro: } 4.97\%$$

$$x: 3.0, y: 35.45656444520654$$

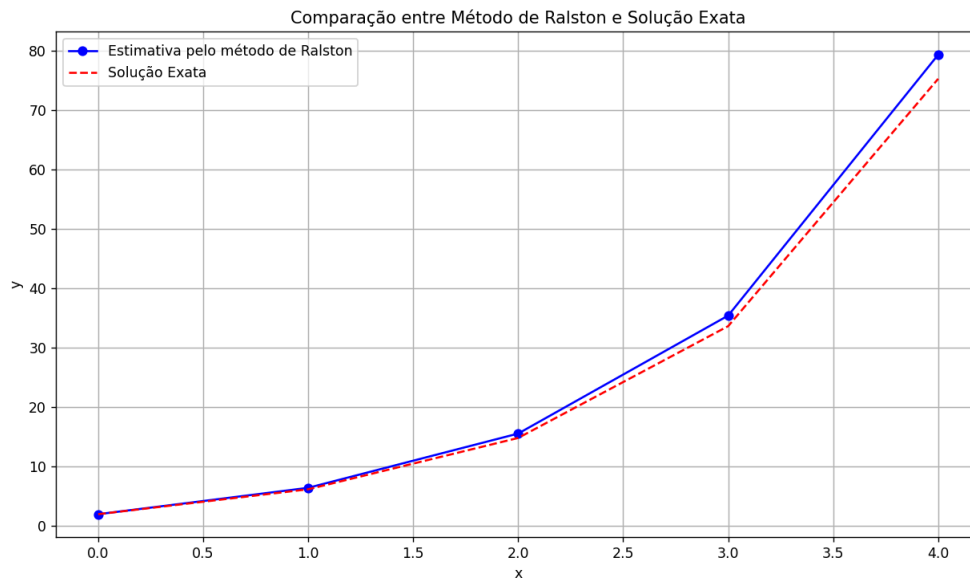
y_exato: 33.6771717679682

Erro: 5.28%

x: 4.0, y: 79.39617670030174

y_exato: 75.3389626091586

Erro: 5.39%



14.Problema 2:

Entrada:

$$f(x) = -2*x**3 + 12*x**2 - 20*x + 8.5$$

$$x0 = 0$$

$$y0 = 1$$

$$h = 0.5$$

$$n = 8$$

Resultado Obtido:

x: 0.0, y: 1.0

y_exato: 1.0000000000000000

Erro: 0%

x: 0.5, y: 3.27734375

y_exato: 3.218750000000000

Erro: 1.82%

x: 1.0, y: 3.1015625

y_exato: 3.000000000000000

Erro: 3.39%

x: 1.5, y: 2.34765625

y_exato: 2.218750000000000

Erro: 5.81%

x: 2.0, y: 2.140625

y_exato: 2.000000000000000

Erro: 7.03%

x: 2.5, y: 2.85546875

y_exato: 2.718750000000000

Erro: 5.03%

x: 3.0, y: 4.1171875

y_exato: 4.000000000000000

Erro: 2.93%

x: 3.5, y: 4.80078125

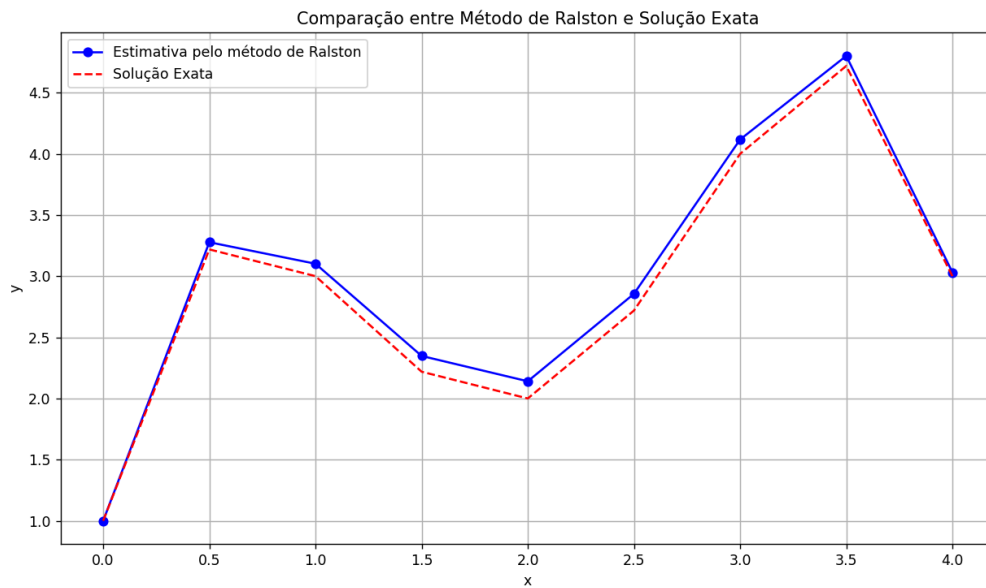
y_exato: 4.718750000000000

Erro: 1.74%

x: 4.0, y: 3.03125

y_exato: 3.000000000000000

Erro: 1.04%



15.Problema 3:

Entrada:

$$f(x) = y^2 + 1$$

$$x_0 = 0$$

$$y_0 = 0$$

$$h = 0.2$$

$$n = 5$$

Resultado Obtido:

x: 0.0, y: 0.0

y_exato: 0

Erro: nan%

x: 0.2, y: 0.20299999999999999

y_exato: 0.202710035508673

Erro: 0.14%

x: 0.4, y: 0.422948765625043

y_exato: 0.422793218738162

Erro: 0.04%

x: 0.6000000000000001, y: 0.6828395408405203

y_exato: 0.684136808341692

Erro: -0.19%

x: 0.8, y: 1.0225924311975236

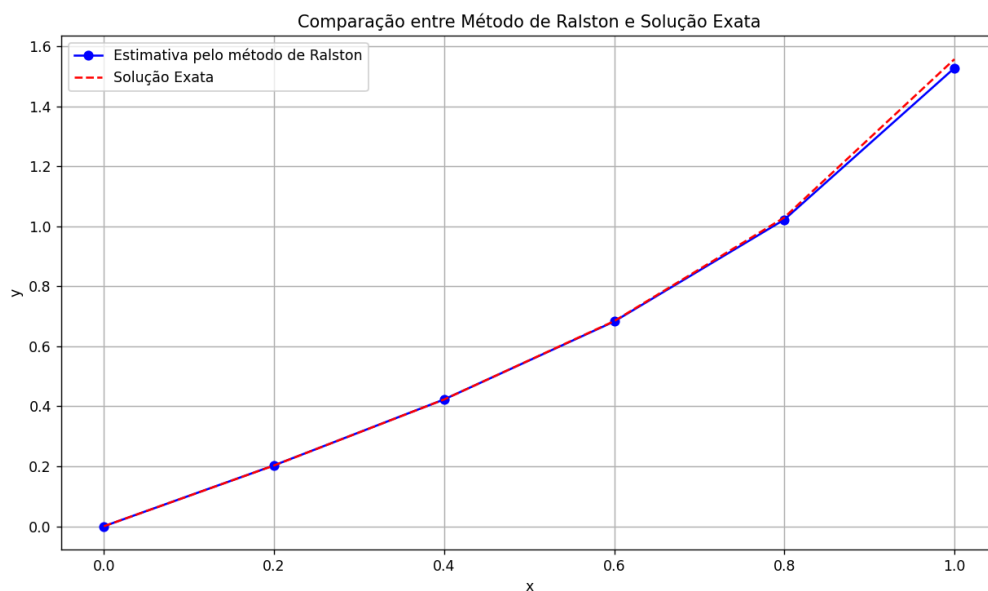
y_exato: 1.02963855705036

Erro: -0.68%

x: 1.0, y: 1.5279625952146503

y_exato: 1.55740772465490

Erro: -1.89%



16.Problema 4:

Entrada:

$$f(x) = -2 \cdot x \cdot y$$

$$x_0 = 0$$

$$y_0 = 1$$

$$h = 0.3$$

$$n = 5$$

Resultado Obtido:

x: 0.0, y: 1.0

y_exato: 1.000000000000000

Erro: 0%

x: 0.3, y: 0.91

y_exato: 0.913931185271228

Erro: -0.43%

x: 0.6, y: 0.6900985000000001

y_exato: 0.697676326071031

Erro: -1.09%

x: 0.8999999999999999, y: 0.4410419513500001

y_exato: 0.444858066222941

Erro: -0.86%

x: 1.2, y: 0.2435654176330376

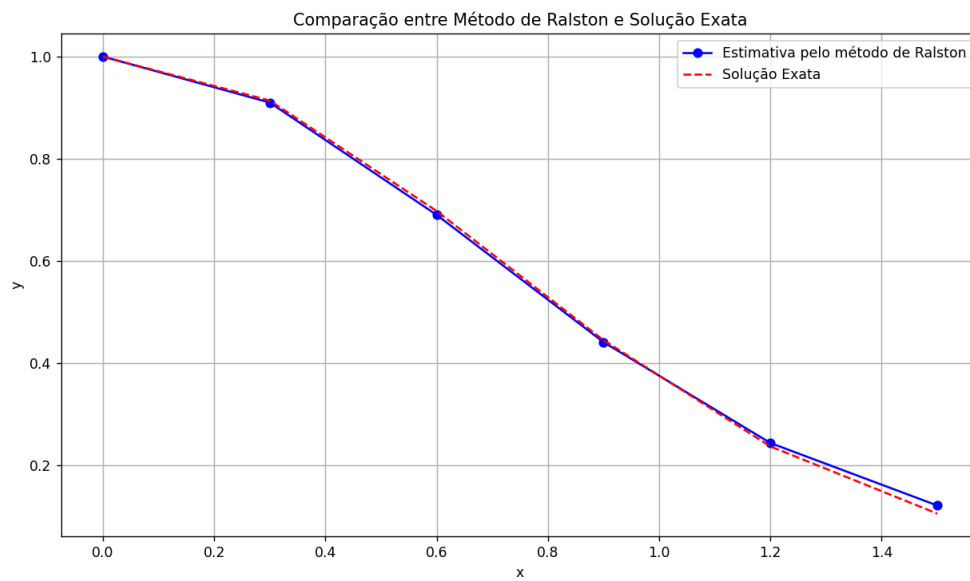
y_exato: 0.236927758682122

Erro: 2.80%

x: 1.5, y: 0.12124686489772611

y_exato: 0.105399224561864

Erro: 15.04%



Dificuldades enfrentadas:

Não houve dificuldades enfrentadas.

6. Método de Hunge-Kutta de 3 ordem

Estratégia de Implementação:

O arquivo contém 4 funções principais: `main()`, `grafico()`, `solucao()` e `hungeKutta()`.

A função `main()` tem como objetivo encontrar o diretório em que a pasta do arquivo se encontra, e definir o arquivo de entrada “in.txt” e o arquivo “out.txt”. Após isso, ela faz a leitura dos dados do arquivo de entrada e faz a chamada das demais funções do algoritmo para a realização dos cálculos necessários para o método. Por fim, o resultado gerado é posto no arquivo de saída.

A função `hungeKutta()` tem como objetivo fazer o cálculo do método e gerar a sua aproximação. Ela tem como parâmetros a função “f”, e os valores de “x0”, “y0”, “h” e “n”.

Por último, a função `grafico()` recebe a função, e os valores de “x0”, “y0”, “h” e “n”, e plota o gráfico que contém o conjunto de dados e a solução correta para o problema. Para a realização do gráfico, foi necessário utilizar a biblioteca Matplotlib.

Estrutura dos arquivos de entrada / saída:

A estrutura do arquivo de entrada “in.txt” consiste em 5 linhas, a primeira contendo a função, seguido do x0, y0, h e n, respectivamente, cada um em uma linha

O arquivo de saída “out.txt” consiste em $3 * (n + 1)$ linhas, para cada divisão n (pontos internos), ele irá printar os valores de x e y em uma linha, o y exato na outra, e a taxa de erro na terceira linha, repetindo-se para todos os pontos internos.

Problema teste:

Para a realização do método, foram testados dois problemas apresentados nos slides do professor (Problema 1 e Problema 2). Além destes 2 exercícios da questão 12.2 da página 389 do livro Neide Franco (Problema 3 e Problema 4:

12.2 - Considere os seguintes problemas de valor inicial:

$$a) \begin{cases} y' = y^2 + 1 \\ y(0) = 0 \end{cases} ; \quad 0 \leq x \leq 1 ; \quad h = 0.2$$

$$b) \begin{cases} y' = -2xy \\ y(0) = 1 \end{cases} ; \quad 0 \leq x \leq 0.6 ; \quad h = 0.3$$

Obs: A quantidade de pontos internos foram aumentados para esses 2 problemas.

17.Problema 1:

Entrada:

$$f(x) = 4*(E)**(0.8*x) - 0.5*y$$

$$x_0 = 0$$

$$y_0 = 2$$

$$h = 1$$

$$n = 4$$

Resultado Obtido:

$$x: 0.0, y: 2.0$$

$$y_{\text{exato}}: 2.0000000000000000$$

$$\text{Erro: } 0\%$$

$$x: 1.0, y: 6.175676680944186$$

$$y_{\text{exato}}: 6.19463137720937$$

$$\text{Erro: } -0.31\%$$

$$x: 2.0, y: 14.786163920719853$$

$$y_{\text{exato}}: 14.8439219076465$$

$$\text{Erro: } -0.39\%$$

$$x: 3.0, y: 33.53672003258817$$

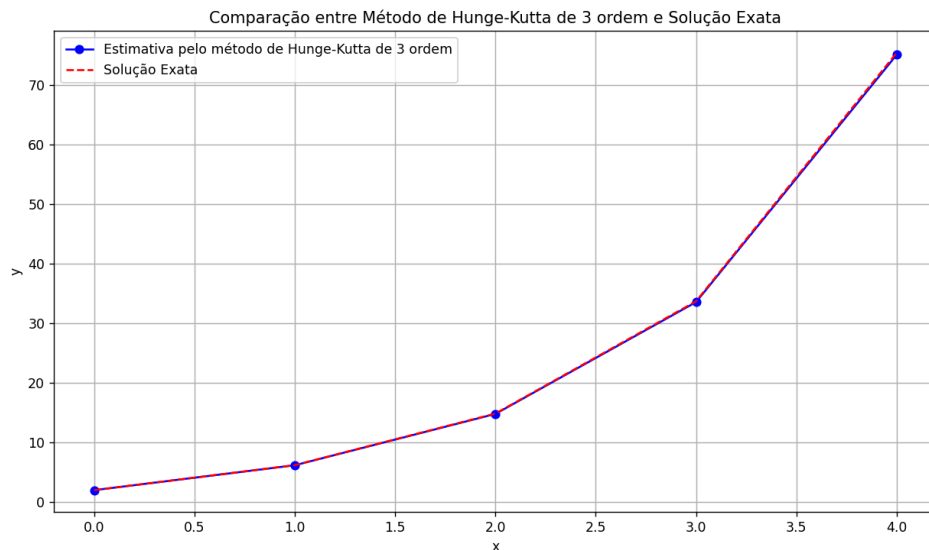
y_exato: 33.6771717679682

Erro: -0.42%

x: 4.0, y: 75.01767021694317

y_exato: 75.3389626091586

Erro: -0.43%



18.Problema 2:

Entrada:

$$f(x) = -2x^3 + 12x^2 - 20x + 8.5$$

$$x_0 = 0$$

$$y_0 = 1$$

$$h = 0.5$$

$$n = 8$$

Resultado Obtido:

$$x: 0.0, y: 1.0$$

$$y_{\text{exato}}: 1.0000000000000000$$

$$\text{Erro: } 0\%$$

$$x: 0.5, y: 3.21875$$

y_exato: 3.218750000000000

Erro: 0%

x: 1.0, y: 3.0

y_exato: 3.000000000000000

Erro: 0%

x: 1.5, y: 2.21875

y_exato: 2.218750000000000

Erro: 0%

x: 2.0, y: 2.0

y_exato: 2.000000000000000

Erro: 0%

x: 2.5, y: 2.71875

y_exato: 2.718750000000000

Erro: 0%

x: 3.0, y: 4.0

y_exato: 4.000000000000000

Erro: 0%

x: 3.5, y: 4.71875

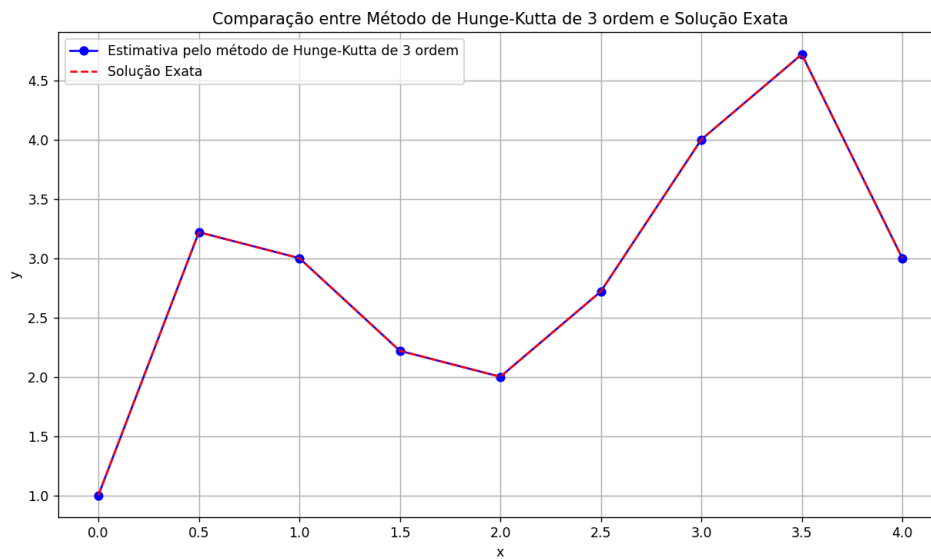
y_exato: 4.718750000000000

Erro: 0%

x: 4.0, y: 3.0

y_exato: 3.000000000000000

Erro: 0%



19.Problema 3:

Entrada:

$$f(x) = y^2 + 1$$

$$x_0 = 0$$

$$y_0 = 0$$

$$h = 0.2$$

$$n = 5$$

Resultado Obtido:

x: 0.0, y: 0.0

y_exato: 0

Erro: nan%

x: 0.2, y: 0.20272053333333337

y_exato: 0.202710035508673

Erro: 0.01%

x: 0.4, y: 0.42286840325523106

y_exato: 0.422793218738162

Erro: 0.02%

x: 0.6000000000000001, y: 0.6843408722447045

y_exato: 0.684136808341692

Erro: 0.03%

x: 0.8, y: 1.0300155495312928

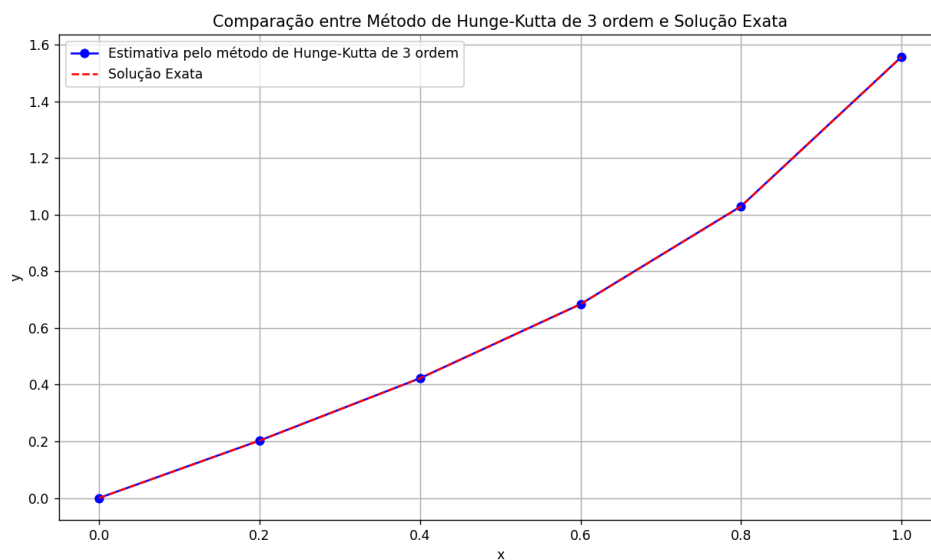
y_exato: 1.02963855705036

Erro: 0.04%

x: 1.0, y: 1.5575649350643381

y_exato: 1.55740772465490

Erro: 0.01%



20.Problema 4:

Entrada:

$$f(x) = -2*x*y$$

$$x_0 = 0$$

$$y_0 = 1$$

$$h = 0.3$$

$$n = 5$$

Resultado Obtido:

x: 0.0, y: 1.0

y_exato: 1.000000000000000

Erro: 0%

x: 0.3, y: 0.9154

y_exato: 0.913931185271228

Erro: 0.16%

x: 0.6, y: 0.7001748136

y_exato: 0.697676326071031

Erro: 0.36%

x: 0.8999999999999999, y: 0.446725534573072

y_exato: 0.444858066222941

Erro: 0.42%

x: 1.2, y: 0.23630708637632533

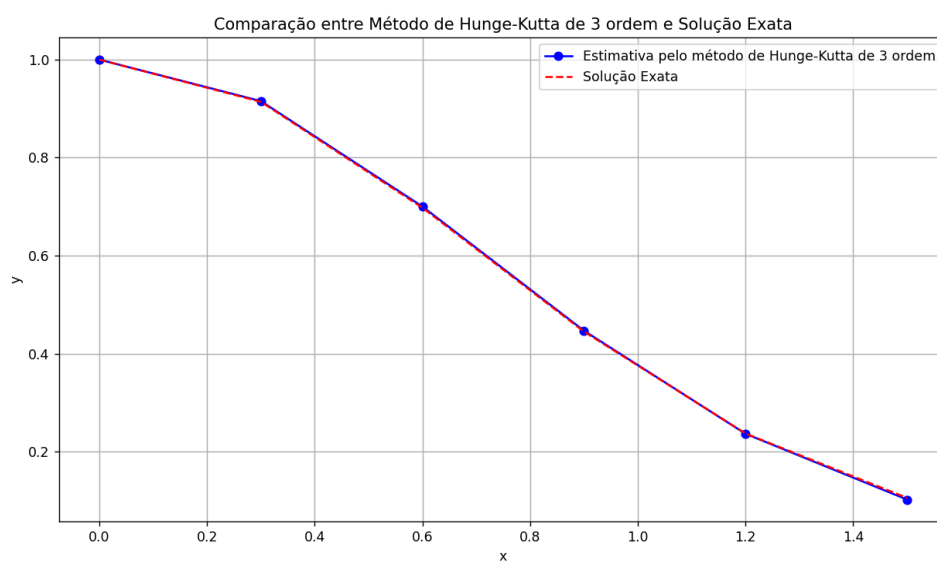
y_exato: 0.236927758682122

Erro: -0.26%

x: 1.5, y: 0.10206575674766244

y_exato: 0.105399224561864

Erro: -3.16%



Dificuldades enfrentadas:

Não houve dificuldades enfrentadas.

7. Método de Hunge-Kutta de 4 ordem

Estratégia de Implementação:

O arquivo contém 4 funções principais: `main()`, `grafico()`, `solucao()` e `hungeKutta()`.

A função `main()` tem como objetivo encontrar o diretório em que a pasta do arquivo se encontra, e definir o arquivo de entrada “in.txt” e o arquivo “out.txt”. Após isso, ela faz a leitura dos dados do arquivo de entrada e faz a chamada das demais funções do algoritmo para a realização dos cálculos necessários para o método. Por fim, o resultado gerado é posto no arquivo de saída.

A função `hungeKutta()` tem como objetivo fazer o cálculo do método e gerar a sua aproximação. Ela tem como parâmetros a função “f”, e os valores de “x0”, “y0”, “h” e “n”.

Por último, a função `grafico()` recebe a função, e os valores de “x0”, “y0”, “h” e “n”, e plota o gráfico que contém o conjunto de dados e a solução correta para o problema. Para a realização do gráfico, foi necessário utilizar a biblioteca Matplotlib.

Estrutura dos arquivos de entrada / saída:

A estrutura do arquivo de entrada “in.txt” consiste em 5 linhas, a primeira contendo a função, seguido do x0, y0, h e n, respectivamente, cada um em uma linha

O arquivo de saída “out.txt” consiste em $3 * (n + 1)$ linhas, para cada divisão n (pontos internos), ele irá printar os valores de x e y em uma linha, o y exato na outra, e a taxa de erro na terceira linha, repetindo-se para todos os pontos internos.

Problema teste:

Para a realização do método, foram testados dois problemas apresentados nos slides do professor (Problema 1 e Problema 2). Além destes 2 exercícios da questão 12.2 da página 389 do livro Neide Franco (Problema 3 e Problema 4:

12.2 - Considere os seguintes problemas de valor inicial:

$$a) \begin{cases} y' = y^2 + 1 \\ y(0) = 0 \end{cases} ; \quad 0 \leq x \leq 1 ; \quad h = 0.2$$

$$b) \begin{cases} y' = -2xy \\ y(0) = 1 \end{cases} ; \quad 0 \leq x \leq 0.6 ; \quad h = 0.3$$

Obs: A quantidade de pontos internos foram aumentados para esses 2 problemas.

21.Problema 1:

Entrada:

$$f(x) = 4*(E)**(0.8*x) - 0.5*y$$

$$x_0 = 0$$

$$y_0 = 2$$

$$h = 1$$

$$n = 4$$

Resultado Obtido:

$$x: 0.0, y: 2.0$$

$$y_{\text{exato}}: 2.0000000000000000$$

$$\text{Erro: } 0\%$$

$$x: 1.0, y: 6.201037072414292$$

$$y_{\text{exato}}: 6.19463137720937$$

$$\text{Erro: } 0.10\%$$

$$x: 2.0, y: 14.8624835881192$$

$$y_{\text{exato}}: 14.8439219076465$$

$$\text{Erro: } 0.13\%$$

$$x: 3.0, y: 33.72134801335574$$

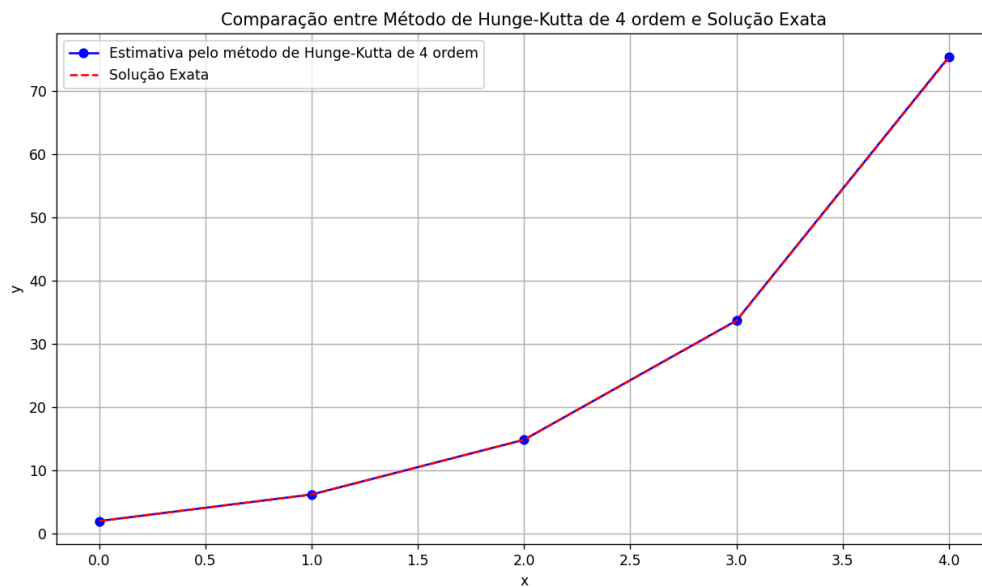
y_exato: 33.6771717679682

Erro: 0.13%

x: 4.0, y: 75.43917199038293

y_exato: 75.3389626091586

Erro: 0.13%



22.Problema 2:

Entrada:

$$f(x) = -2*x**3 + 12*x**2 - 20*x + 8.5$$

$$x_0 = 0$$

$$y_0 = 1$$

$$h = 0.5$$

$$n = 8$$

Resultado Obtido:

$$x: 0.0, y: 1.0$$

$$y_{\text{exato}}: 1.0000000000000000$$

Erro: 0%

$$x: 0.5, y: 3.21875$$

y_exato: 3.218750000000000

Erro: 0%

x: 1.0, y: 3.0

y_exato: 3.000000000000000

Erro: 0%

x: 1.5, y: 2.21875

y_exato: 2.218750000000000

Erro: 0%

x: 2.0, y: 2.0

y_exato: 2.000000000000000

Erro: 0%

x: 2.5, y: 2.71875

y_exato: 2.718750000000000

Erro: 0%

x: 3.0, y: 4.0

y_exato: 4.000000000000000

Erro: 0%

x: 3.5, y: 4.71875

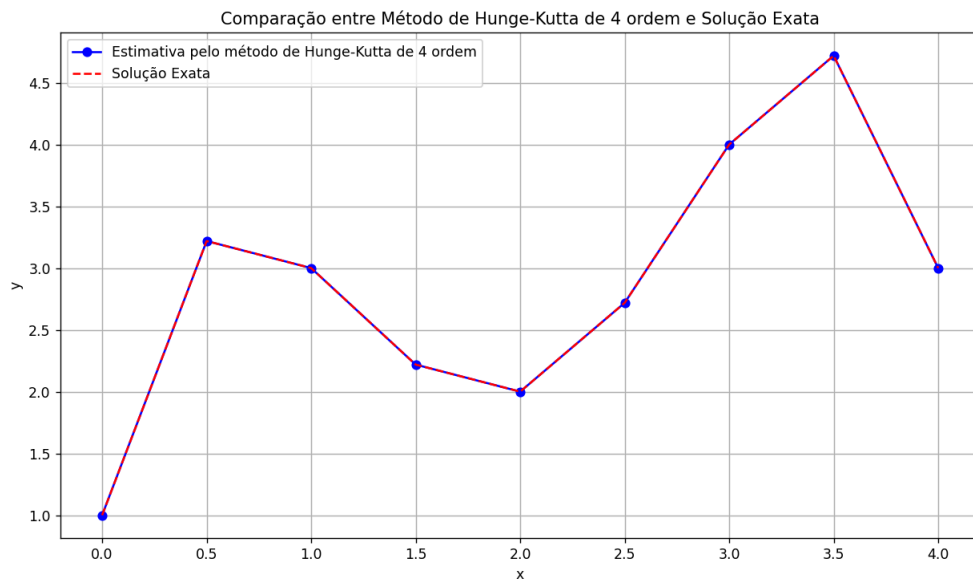
y_exato: 4.718750000000000

Erro: 0%

x: 4.0, y: 3.0

y_exato: 3.000000000000000

Erro: 0%



23.Problema 3:

Entrada:

$$f(x) = y^*2+1$$

$$x_0 = 0$$

$$y_0 = 0$$

$$h = 0.2$$

$$n = 5$$

Resultado Obtido:

x: 0.0, y: 0.0

y_exato: 0

Erro: nan%

x: 0.2, y: 0.20270740808053467

y_exato: 0.202710035508673

Erro: 0.0%

x: 0.4, y: 0.4227889928132791

y_exato: 0.422793218738162

Erro: 0.0%

x: 0.6000000000000001, y: 0.6841334019907197

y_exato: 0.684136808341692

Erro: 0.0%

x: 0.8, y: 1.029636624958974

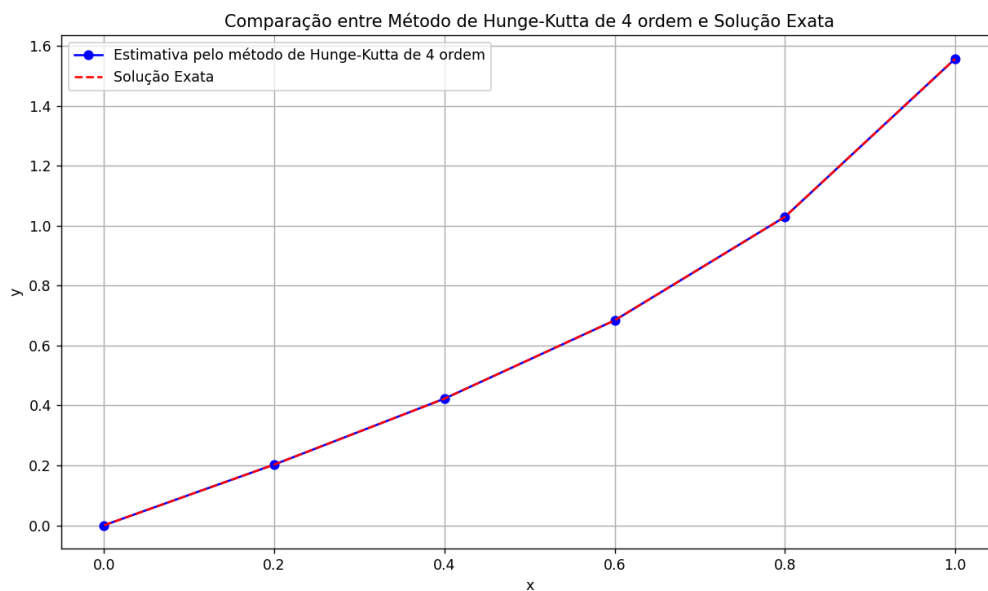
y_exato: 1.02963855705036

Erro: 0.0%

x: 1.0, y: 1.5573516911576377

y_exato: 1.55740772465490

Erro: 0.0%



24.Problema 4:

Entrada:

$$f(x) = -2 \cdot x \cdot y$$

$$x_0 = 0$$

$$y_0 = 1$$

$$h = 0.3$$

$$n = 5$$

Resultado Obtido:

x: 0.0, y: 1.0

y_exato: 1.000000000000000

Erro: 0%

x: 0.3, y: 0.9139285

y_exato: 0.913931185271228

Erro: 0.0%

x: 0.6, y: 0.697662244927405

y_exato: 0.697676326071031

Erro: 0.0%

x: 0.8999999999999999, y: 0.4449011321613798

y_exato: 0.444858066222941

Erro: 0.01%

x: 1.2, y: 0.23722363274642558

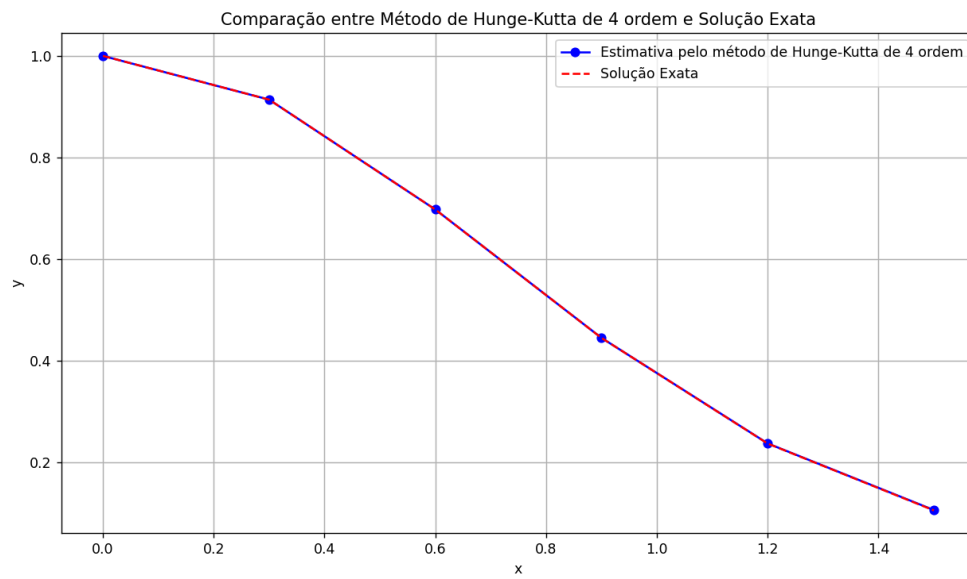
y_exato: 0.236927758682122

Erro: 0.12%

x: 1.5, y: 0.10608432099623336

y_exato: 0.105399224561864

Erro: 0.65%



Dificuldades enfrentadas:

Não houve dificuldades enfrentadas.

8. Método de Sistemas de Equações Diferenciais

Estratégia de Implementação:

O arquivo contém 4 funções principais: `main()`, `grafico()`, `solucao()` e `hungeKuttaSistemasDiferenciais()`.

A função `main()` tem como objetivo encontrar o diretório em que a pasta do arquivo se encontra, e definir o arquivo de entrada “in.txt” e o arquivo “out.txt”. Após isso, ela faz a leitura dos dados do arquivo de entrada e faz a chamada das demais funções do algoritmo para a realização dos cálculos necessários para o método. Por fim, o resultado gerado é posto no arquivo de saída.

A função `hungeKuttaSistemasDiferenciais()` tem como objetivo fazer o cálculo do método e gerar a sua aproximação. Ela tem como parâmetros as funções “f1” e “f2”, e os valores de “x0”, “y0”, “y1”, “h” e “n”.

Por último, a função `grafico()` recebe o resultado do método e as soluções exatas para y e z, e plota o gráfico que contém o conjunto de dados e a solução correta para o problema. Para a realização do gráfico, foi necessário utilizar a biblioteca Matplotlib.

Estrutura dos arquivos de entrada / saída:

A estrutura do arquivo de entrada “in.txt” consiste em 7 linhas, as duas primeiras contendo as funções f1 e f2, seguido do x0, y0, y1, h e n, respectivamente, cada um em uma linha

O arquivo de saída “out.txt” consiste em $3 * (n + 1)$ linhas, para cada divisão n (pontos internos), ele irá printar os valores de x, y1 e y2 em uma linha, o y1 e y2 exatos na outra, e a taxa de erro na terceira linha, repetindo-se para todos os pontos internos.

Problema teste:

Para a realização do método, foi utilizado o exemplo dado no slide do professor.

25.Problema 1:

Entrada:

$$f1 = -0.5 * y$$

$$f2 = 4 - 0.3 * z - 0.1 * y$$

$$x0 = 0$$

$$y0 = 4$$

$$y1 = 6$$

$$h = 0.5$$

$$n = 4$$

Resultado Obtido:

x: 0.0, y1: 4.0, y2: 6.0

y1_exato: 4.000000000000000, y2_exato: 6.000000000000000

Erro y1: 0%, Erro y2: 0%

x: 0.5, y1: 3.115234375, y2: 6.8576703125

y1_exato: 3.11520313228562, y2_exato: 6.85766045284227

Erro y1: 0.0%, Erro y2: 0.0%

x: 1.0, y1: 2.42617130279541, y2: 7.632105673416747

y1_exato: 2.42612263885053, y2_exato: 7.63209125972923

Erro y1: 0.0%, Erro y2: 0.0%

x: 1.5, y1: 1.8895230605266988, y2: 8.326885976668454

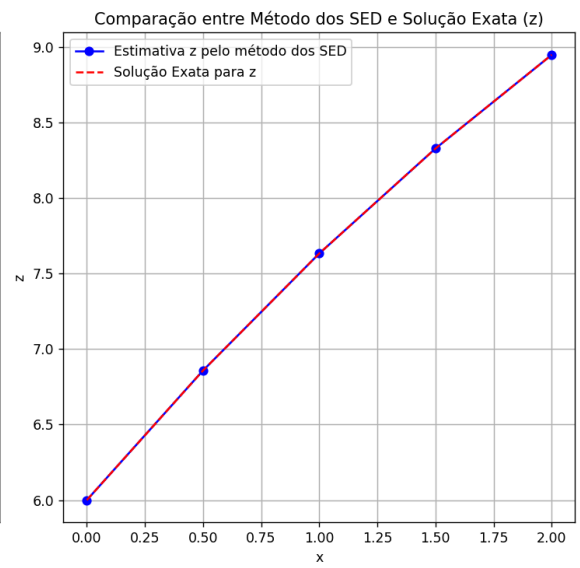
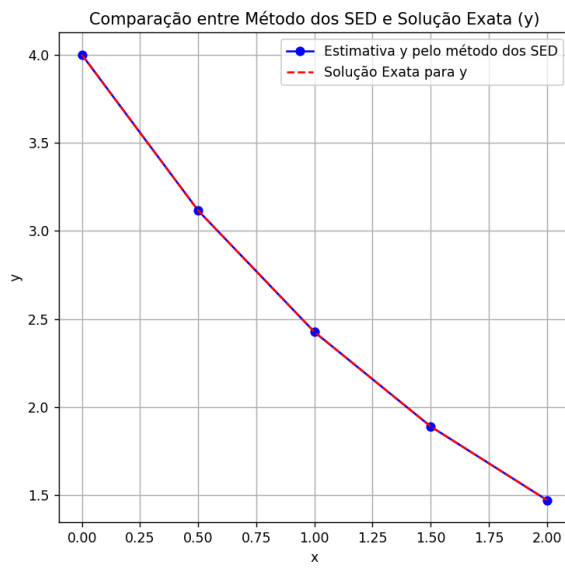
y1_exato: 1.88946621096406, y2_exato: 8.32687035701215

Erro y1: 0.0%, Erro y2: 0.0%

x: 2.0, y1: 1.4715767976269944, y2: 8.946865099966331

y1_exato: 1.47151776468577, y2_exato: 8.94685027879864

Erro y1: 0.0%, Erro y2: 0.0%



Dificuldades enfrentadas:

Não houve dificuldades enfrentadas.

9. Método de Shooting

Estratégia de Implementação:

O arquivo contém 5 funções principais: `main()`, `grafico()`, `solucao()`, `shooting()` e `hungeKutta()`, essa última apenas para ser chamada na função do método de Shooting.

A função `main()` tem como objetivo encontrar o diretório em que a pasta do arquivo se encontra, e definir o arquivo de entrada “in.txt” e o arquivo “out.txt”. Após isso, ela faz a leitura dos dados do arquivo de entrada e faz a chamada das demais funções do algoritmo para a realização dos cálculos necessários para o método. Por fim, o resultado gerado é posto no arquivo de saída.

A `shooting()` tem como objetivo fazer o cálculo do método e gerar a sua aproximação. Ela tem como parâmetros a função “f”, e os valores de “x0”, y0”, “h”, “n” e “yb”.

Por último, a função `grafico()` recebe a função, e os valores de “x0”, y0”, “h”, “n” e “yb”, além do resultado gerado pelo método de Shooting, e plota o gráfico que contém o conjunto de dados e a solução correta para o problema. Para a realização do gráfico, foi necessário utilizar a biblioteca Matplotlib.

Estrutura dos arquivos de entrada / saída:

A estrutura do arquivo de entrada “in.txt” consiste em 6 linhas, as duas primeiras contendo a função f, seguido do x0, y0, h, n e yb, respectivamente, cada um em uma linha

O arquivo de saída “out.txt” consiste em $3 * (n + 1)$ linhas, para cada divisão n (pontos internos), ele irá printar os valores de x e y em uma linha, o y exato na outra, e a taxa de erro na terceira linha, repetindo-se para todos os pontos internos.

Problema teste:

Para a realização do método, foi utilizado este problema:

$$f(x) = -x*y + 4*x/y$$

$$x_0 = 0$$

$$y_0 = 1$$

$$h = 0.1$$

$$n = 10$$

$$y_b = 1.4$$

26.Problema 1:

Resultado Obtido:

x: 0.0, y: 1.0

y_exato: 1.0000000000000000

Erro: 0.0%

x: 0.1, y: 1.0148158670028387

y_exato: 1.01481549985822

Erro: 0.0%

x: 0.2, y: 1.057182198104166

y_exato: 1.05718100746420

Erro: 0.0%

x: 0.30000000000000004, y: 1.1216998897177695

y_exato: 1.12169801826798

Erro: 0.0%

x: 0.4, y: 1.2014881036244773

y_exato: 1.20148601036440

Erro: 0.0%

x: 0.5, y: 1.2898071487561804

y_exato: 1.28980527630561

Erro: 0.0%

x: 0.6, y: 1.3809325464253681

y_exato: 1.38093121544373

Erro: 0.0%

x: 0.7, y: 1.470415759334389

y_exato: 1.47041518539722

Erro: 0.0%

x: 0.7999999999999999, y: 1.5550310971382222

y_exato: 1.55503142343518

Erro: -0.0%

x: 0.8999999999999999, y: 1.6326118664063782

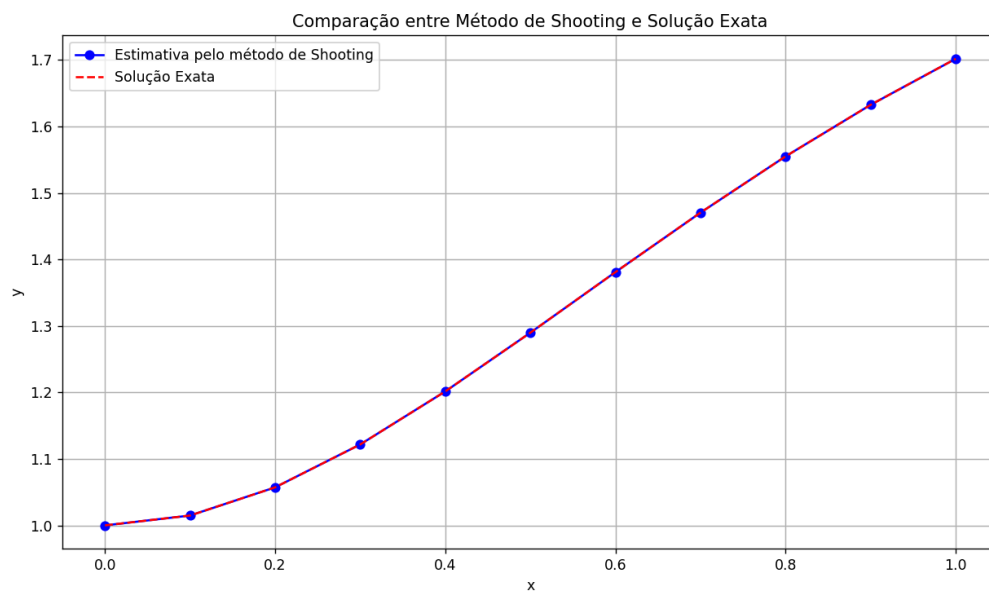
y_exato: 1.63261318178287

Erro: -0.0%

x: 0.9999999999999999, y: 1.7018677085421234

y_exato: 1.70187005276128

Erro: -0.0%



Dificuldades enfrentadas:

Não houve dificuldades enfrentadas.

10. Método das diferenças finitas

Estratégia de Implementação:

O arquivo contém 4 funções principais: `main()`, `grafico()`, `solucao()` e `diferencasFinitas()`.

A função `main()` tem como objetivo encontrar o diretório em que a pasta do arquivo se encontra, e definir o arquivo de entrada “in.txt” e o arquivo “out.txt”. Após isso, ela faz a leitura dos dados do arquivo de entrada e faz a chamada das demais funções do algoritmo para a realização dos cálculos necessários para o método. Por fim, o resultado gerado é posto no arquivo de saída.

A `diferencasFinitas()` tem como objetivo fazer o cálculo do método e gerar a sua aproximação. Ela tem como parâmetros a função “f”, e os valores de “x0”, y0”, “h”, “n” e “yb”.

Por último, a função `grafico()` recebe a função, e os valores de “x0”, y0”, “h”, “n” e “yb”, além do resultado gerado pelo método das diferenças finitas, e plota o gráfico que contém o conjunto de dados e a solução correta para o problema. Para a realização do gráfico, foi necessário utilizar a biblioteca Matplotlib.

Estrutura dos arquivos de entrada / saída:

A estrutura do arquivo de entrada “in.txt” consiste em 6 linhas, as duas primeiras contendo a função f, seguido do x0, y0, h, n e yb, respectivamente, cada um em uma linha

O arquivo de saída “out.txt” consiste em $3 * (n + 1)$ linhas, para cada divisão n (pontos internos), ele irá printar os valores de x e y em uma linha, o y exato na outra, e a taxa de erro na terceira linha, repetindo-se para todos os pontos internos.

Problema teste:

Para a realização do método, foi utilizado este problema:

$$f(x) = -x*y + 4*x/y$$

$$x0 = 0$$

$$y0 = 1$$

$$h = 0.1$$

$$n = 10$$

$$y_b = 1.4$$

27.Problema 1:

Resultado Obtido:

x: 0.0, y: 1.0

y_exato: 1.0000000000000000

Erro: 0.0%

x: 0.1, y: 1.0

y_exato: 1.01481549985822

Erro: -1.46%

x: 0.2, y: 1.03

y_exato: 1.05718100746420

Erro: -2.57%

x: 0.30000000000000004, y: 1.0870699029126214

y_exato: 1.12169801826798

Erro: -3.09%

x: 0.4, y: 1.1648462911273196

y_exato: 1.20148601036440

Erro: -3.05%

x: 0.5, y: 1.255609618037652

y_exato: 1.28980527630561

Erro: -2.65%

x: 0.6000000000000001, y: 1.3521143139350462

y_exato: 1.38093121544373

Erro: -2.09%

x: 0.7000000000000001, y: 1.4484872400849704

y_exato: 1.47041518539722

Erro: -1.49%

x: 0.8, y: 1.5403982534425766

y_exato: 1.55503142343518

Erro: -0.94%

x: 0.9, y: 1.6249048785133102

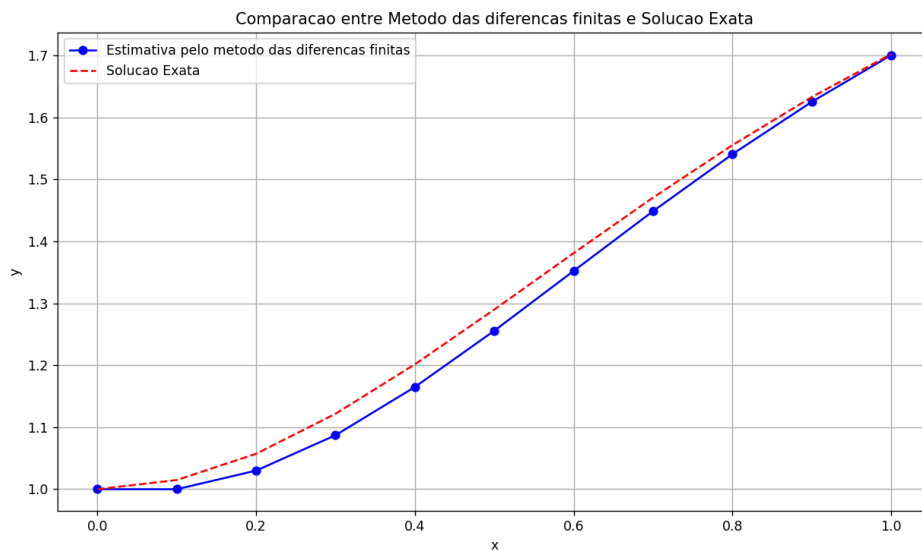
y_exato: 1.63261318178287

Erro: -0.47%

x: 1.0, y: 1.7002148697864552

y_exato: 1.70187005276128

Erro: -0.1%



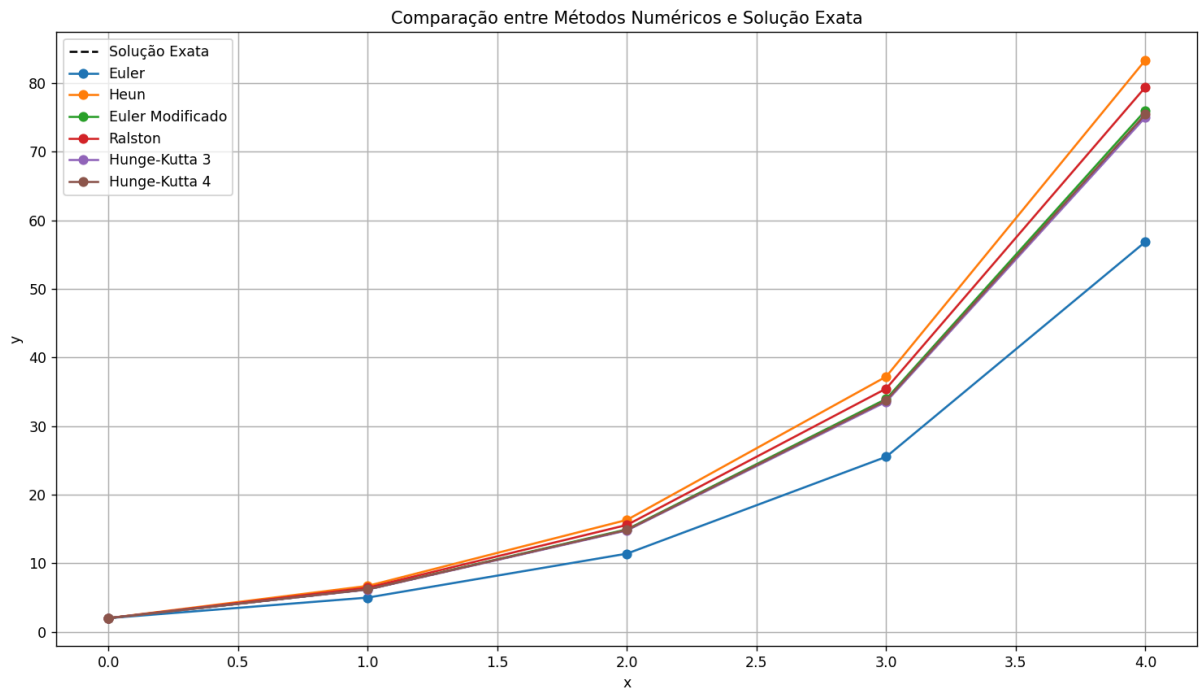
Dificuldades enfrentadas:

Não houve dificuldades enfrentadas.

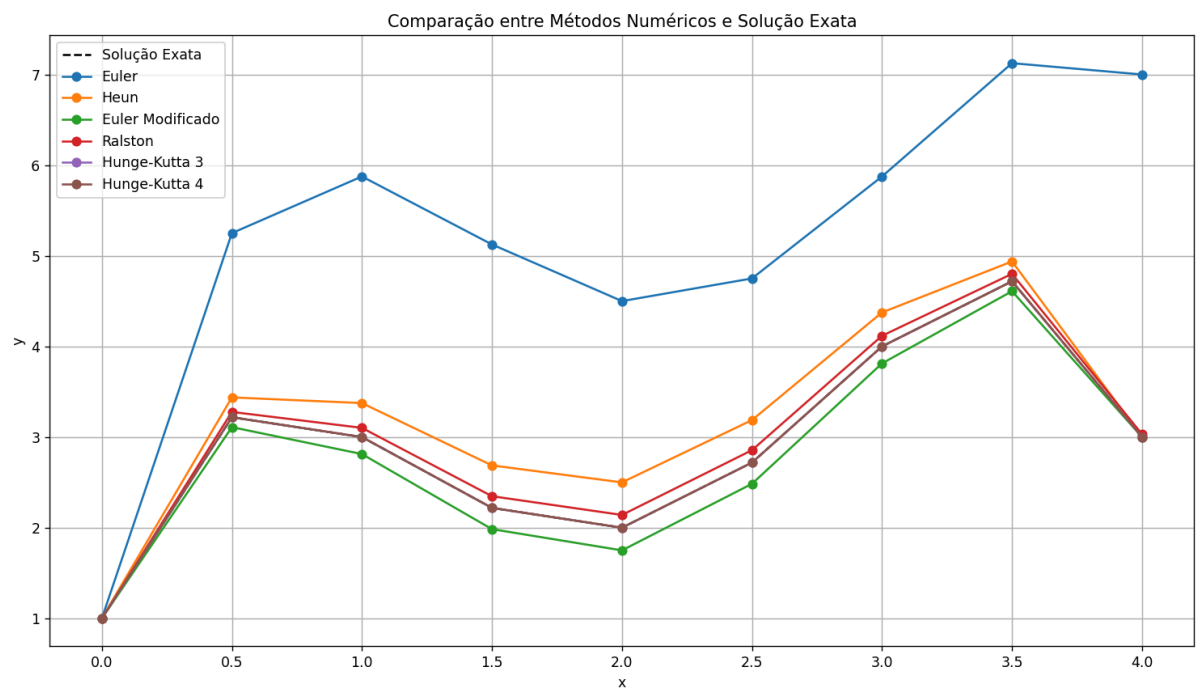
11. Comparação de resultados

Comparação dos tópicos 2, 3, 4, 5, 6, 7:

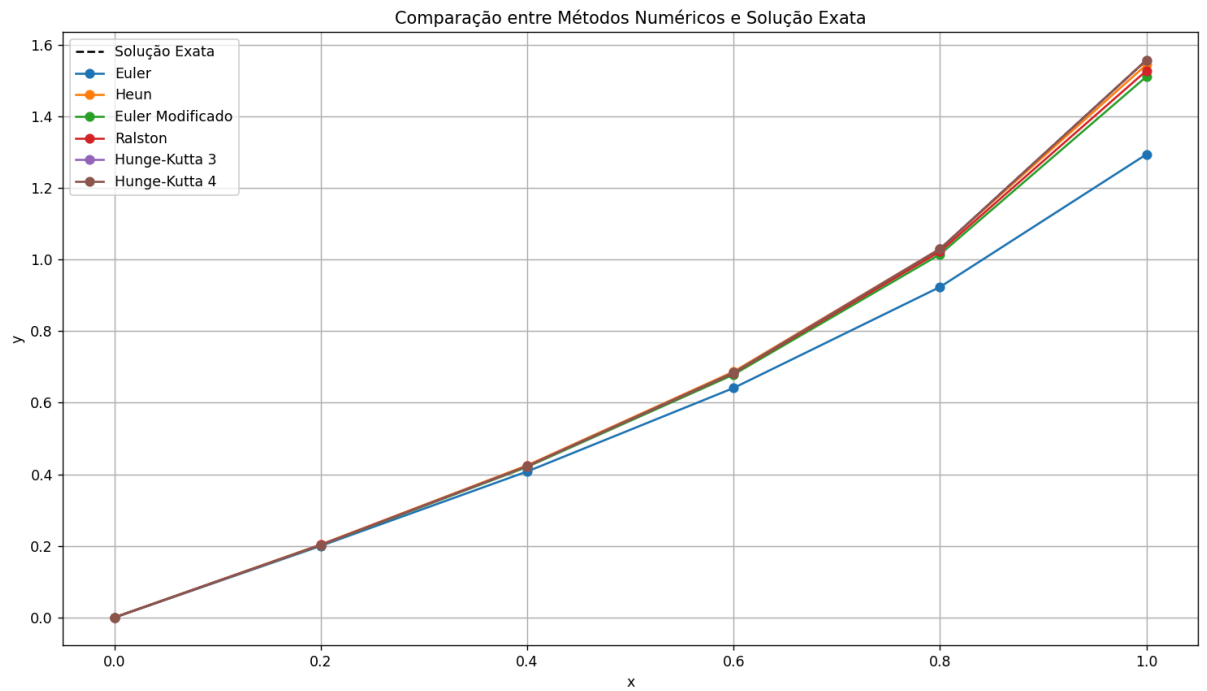
1. Problema 1:



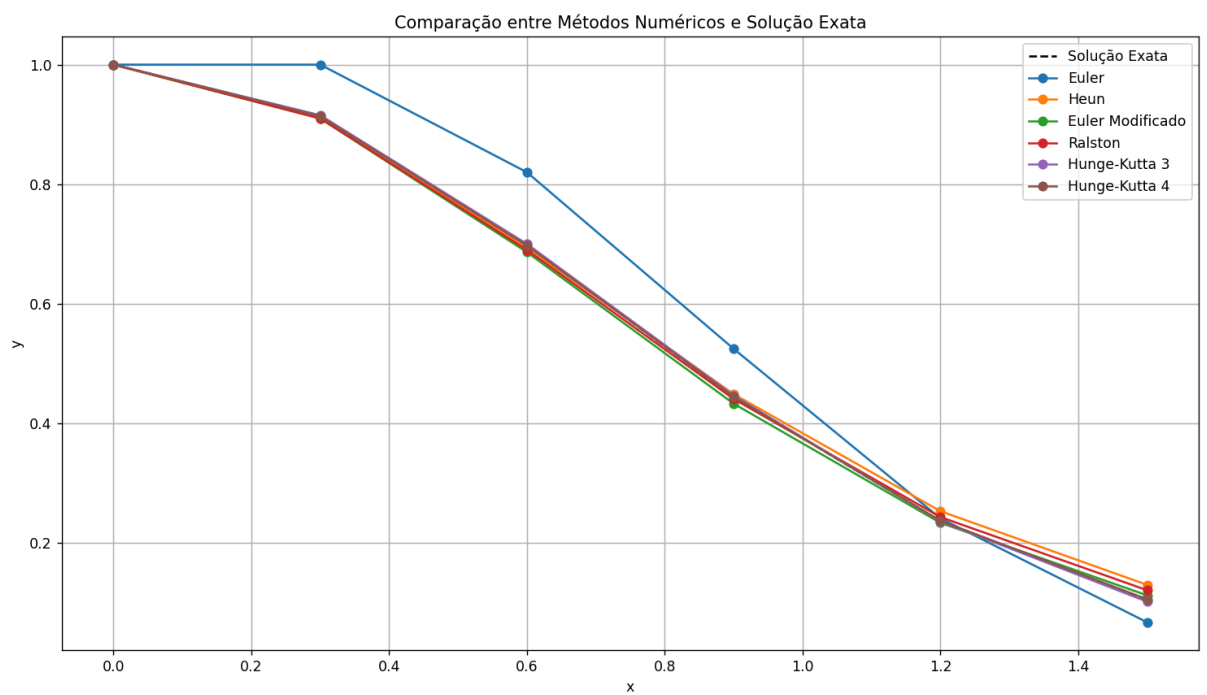
2. Problema 2:



3. Problema 3:

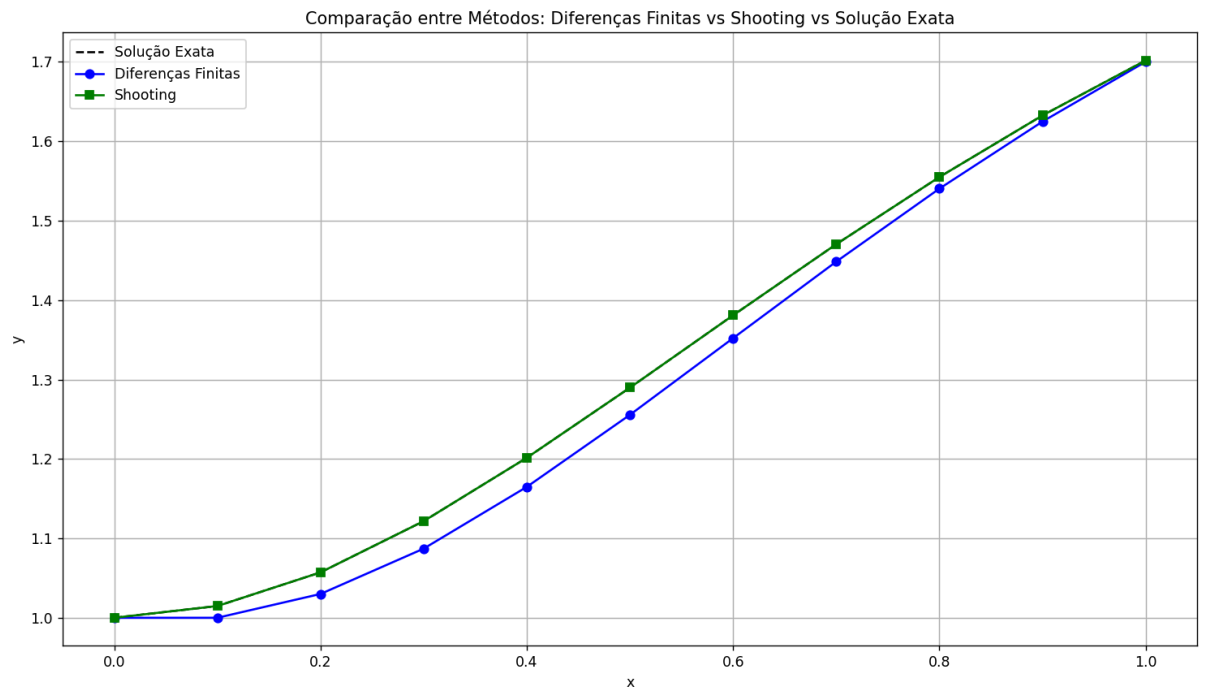


4. Problema 4:



Comparação dos tópicos 9 e 10:

1. Problema 1:



12. Considerações finais

Após concluir este relatório de análise numérica, é evidente a importância dos métodos numéricos na resolução de problemas matemáticos. A implementação de algoritmos e a manipulação de dados permitem alcançar resultados precisos e eficientes. Os testes realizados confirmaram que a maioria das implementações utilizadas conseguiram encontrar estimativas de soluções satisfatórias e com pouca margem de erro.

Assim sendo, mesmo com tais resultados, é necessário que futuras implementações sejam desenvolvidas para possíveis melhorias. É essencial buscar constantemente aprimoramentos nos algoritmos utilizados e incorporar novas funcionalidades para garantir a eficácia dos métodos numéricos empregados. Somente assim será possível resolver problemas cada vez mais complexos e desafiadores com confiabilidade